



Recent progress in exact geometric computation

Chen Li, Sylvain Pion, Chee Yap

► To cite this version:

Chen Li, Sylvain Pion, Chee Yap. Recent progress in exact geometric computation. Journal of Logic and Algebraic Programming, 2005, Practical development of exact real number computation, 64 (1), pp.85-111. <10.1016/j.jlap.2004.07.006>. <inria-00344355>

HAL Id: inria-00344355

<https://inria.hal.science/inria-00344355v1>

Submitted on 4 Dec 2008

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



HAL Authorization

Recent Progress in Exact Geometric Computation [★]

C. Li

*Courant Institute of Mathematical Sciences
New York University, New York, NY 10012, USA.*

S. Pion

*Max Planck Institut für Informatik
Saarbrücken, Germany.*

C. K. Yap

*Courant Institute of Mathematical Sciences
New York University, New York, NY 10012, USA.*

Abstract

Computational geometry has produced an impressive wealth of efficient algorithms. The robust implementation of these algorithms remains a major issue. Among the many proposed approaches for solving numerical non-robustness, Exact Geometric Computation (EGC) is one of the most promising. This survey describes recent progress in EGC research. We specifically focus on the problems of constructive root bounds, precision-driven computation and numerical filters.

Key words: Exact Geometric Computation, Constructive Root Bounds, Arithmetic Filters, Interval Arithmetic, C++ Libraries.

[★] This paper is based on a talk presented at the DIMACS Workshop on Algorithmic and Quantitative Aspects of Real Algebraic Geometry in Mathematics and Computer Science, March 12 – 16, 2001. The work is supported by NSF/ITR Grant #CCR-0082056 and by the IST Programme of the EU as a Shared-cost RTD (FET Open) Project under Contract No IST-2000-26473 (ECG - Effective Computational Geometry for Curves and Surfaces).

Email addresses: chenli@cs.nyu.edu (C. Li), Sylvain.Pion@mpi-sb.mpg.de (S. Pion), yap@cs.nyu.edu (C. K. Yap).

1 Introduction

Numerical non-robustness represents a major challenge in the implementation of geometric algorithms. By its nature, geometric computation has two components, a numerical part and a combinatorial part. Numerical computation is involved in both the construction of new geometric objects and in the evaluation of geometric predicates. An example of the former is computing intersection points and an example of the latter is deciding whether a point is on a hyperplane. Geometric predicates are especially critical, as they determine the combinatorial relations among objects. Incorrect evaluation of such predicates can lead to inconsistencies. In general, computational geometry algorithms are designed under a Real RAM model of computation where all numerical computations are exact. As machine arithmetic is widely used as substitute for this assumed exact arithmetic, numerical errors are inevitable in implementation. Today, machine arithmetic has converged to the IEEE standard [89,31]. But more generally, machine arithmetic is an example of fixed-precision arithmetic. Although numerical errors can sometimes be tolerated and interpreted as small perturbations in inputs, serious problems arise when such errors lead to invalid combinatorial structures or inconsistent state during a program execution.

The numerical non-robustness problem has received much attention in the computational geometry community in the last 15 years ([25,41,88,53,18,26]). The next section will briefly review some approaches to non-robustness, we refer the reader to the current surveys ([93,74,68]) for more details. In [93], robustness literature was classified along two lines: the papers that aim to make fixed-precision algorithms computation robust, and those that aim to make the exact computation approach efficient. Call these the **inexact** and **exact** approaches, respectively, and the corresponding algorithms **Type I** and **Type II** algorithms. Our review below will classify the literature into those that try to make the arithmetic more robust, and those that achieve robustness by ensuring certain geometric (which subsumes topological) properties of problem at hand. Call these the **arithmetic** and the **geometric approaches**, respectively, and the corresponding algorithms **Type A** and **Type B** algorithms. The I-versus-II taxonomy is orthogonal to the A-versus-B taxonomy. For instance, both Type I and Type II algorithms can benefit from Type A development: an example of this is the use of machine arithmetic with scalar product primitive, as proposed by Ottmann et al [66]. Similarly, the introduction of low-degree predicates [54] in the development of Type II algorithms can be useful for Type I as well as Type II algorithms. On the other hand, there is a strong correlation between Type I algorithms and Type B algorithms: if an algorithm computes in the fixed-precision model, then it is very likely that it has to investigate the geometric properties of the computational problem.

Paper Outline. In Section 2, we review the arithmetic and geometric approaches to non-robustness. Then we focus on some issues in Exact Geometric Computation (EGC). In Section 3, we discuss in detail the constructive root bound problem which is a key problem in EGC. We treat the techniques of precision-driven computation and numerical filtering in Sections 4 and 5, respectively. We conclude in Section 6.

2 Literature Review

We review some robustness techniques developed under the Arithmetic and Geometric Approaches. We also single out the Exact Geometric Computation approach which is the focus of this paper.

2.1 Arithmetic Approaches

This is a natural first place to look for a solution since we know that the root cause of numerical non-robustness is errors from approximate arithmetic. The “naive” arithmetic solution says that we just have to compute exactly, without any errors. This requires the use of multi-precision (i.e., unbounded precision) arithmetic. Such arithmetic is implemented in software libraries called “big number packages” (big integers, big rationals, big floats, big complex, etc). For a survey of multi-precision number packages, see [96]. All big number packages support the four basic arithmetic operations ($+$, $-$, $\times$, $\div$). Within the domain of rational numbers, it is clear that these operations can be performed without errors. However, if we need irrational numbers (for example when we take square roots), then simple big number packages will still be insufficient. The usual understanding of “number representation” is that it is some form of positional number system (basically, strings of digits to some base). In this sense, the algebraic number $\sqrt{2}$ cannot be represented exactly. However, it is well-known (e.g., in the computer algebra community) that we can represent and perform all the usual arithmetic operations on algebraic quantities as well. Furthermore, by appeal to a general result that goes back to Tarski, any problem that is algebraic [94] can be computed without errors. This general result forms the backdrop for the development of type I.

The problem with the above naive view of exact arithmetic is the inefficiency of algebraic computations. The complexity of each operation depends on the bit length of operands. In cascaded computations, the bit length of numbers increases quickly. Even for rational operations, the worst-case complexity is exponential in the number of operations. For an instance, Yu [97] concluded that exact rational arithmetic for 3-D polyhedral modeling is impracticable.

Karasick et al [49] reported that the naive use of rational arithmetic in the divide-and-conquer algorithm for 2-D Delaunay triangulation costs a performance penalty of 10^4 over the corresponding floating-point implementation.

Approximate arithmetic. Approximate arithmetic is used by all Type I approaches, mostly in the form of machine arithmetic. But it is increasingly used in Type II approaches as well. An early example of Type II algorithm based on approximate arithmetic is [23] where it is applied to Fortune’s sweepline algorithm using big floats, combined with root bounds. Approximate arithmetic for Type II algorithms requires multi-precision packages, typically some form of big float. This also opens up the possibility of adaptive precision arithmetic. Here, it is helpful to have arithmetic systems that is equipped with the ability to track errors, and to increase precision on demand [96]. Error tracking amounts to forward error analysis; it is also a form of significance arithmetic [2,58]. More generally, significance arithmetic can be viewed as a form of interval arithmetic [62,1].

It is important to see how approximate arithmetic solves the kind of efficiency bottleneck noted under the naive use of exact arithmetic. The latter ultimately reduces to big rational computation. Rational computations tend to be very slow for various reasons, including the phenomenon of exponential growth of bit sizes and the presence of non-trivial content in fractions (which requires the GCD computation to remove). On the other hand, big floats arithmetic has complexity that is intermediate between big rationals and big integers (we may take big integer arithmetic as the base reference for multi-precision arithmetic). In fact, big float arithmetic is basically big integer arithmetic, plus overhead. In addition to these, there is the possibility of adaptive precision computation.

Other ideas to improve robustness at the arithmetic level (without necessarily completely eliminating non-robustness) are to improve the accuracy or range of fixed precision arithmetic [19,56]. More low level techniques here include providing accurate scalar products [66] and machine architectures that provide a “fused multiplication and add” (FMA) primitive. Programming language support for robustness can be valuable for programmers. For instance, special facilities for robust arithmetic has been incorporated into programming languages (e.g., Numerical Turing [47], Pascal-SC [7]).

2.2 Geometric Approaches

The geometric approach lends itself to considerably more diverse forms than the arithmetic approach. Hence our review here only touches on some major

representatives. The general idea is to ensure that certain geometric properties are preserved by our algorithms. This is often enough to ensure no inconsistent states in the algorithm (hence robust). For instance, if we are computing the Voronoi diagram of a planar point set, we want to ensure that the output is a planar graph [84]. We stress that in our classification, purely topological approaches is subsumed under geometric approaches (see below for a more formal expression of this). The first decision facing the robust algorithm designer is the choice of which properties to preserve, and this is dictated by efficiency considerations and the needs of the application; as a corollary, the algorithm gives up some other properties.

Finite Resolution Geometry. If we compute in fixed precision arithmetic, then one approach is to invent novel “finite resolution geometries” [35] as a substitute for the standard Euclidean geometry. This ersatz geometry can only preserve a few of the properties found in Euclidean geometry. A very natural and popular finite precision geometry is the grid (usually regular, but this is not essential). Greene and Yao [35] investigated line arrangement computation in this geometry. See also [38,36]. Here, line segment may become polygonal lines so that their intersections preserve properties such as non-braiding and connected intersection. But we give up the properties such as the intersection of two lines is a single point.

Approximate Predicates and Fat Geometry. Another approach to robustness is to focus on the imprecise nature of predicate evaluations. In Epsilon Geometry [37], Guibas et al introduced “epsilon-predicates” which return a real number instead of the usual $-1, 0, +1$. A return value of $\epsilon > 0$ means that there is a perturbation of the input by at most ϵ such that the exact predicate becomes true. If $\epsilon < 0$, this means that any perturbation of the input up to ϵ will still satisfy the exact predicate. Designing robust algorithms in this setting is difficult.

In terms of geometry, we can think of the imprecise predicates as inducing “fat objects” (so a point becomes a ball, and a line becomes a cylinder, etc). Indeed, this is basically what happens in the widespread programming trick called epsilon tweaking [94]. See also [77] for another example. Milenkovic [61] proposed to perturb objects so as to guarantee a minimum separation distance (“well separated”). This ensures that approximate predicates actually yield correct decisions.

Consistency and Topological Approach. The question “what is geometry?” has been asked many times throughout the history of mathematics. In geometric computing, the formula “Geometry = Combinatorics + Numerics”

is particularly suggestive (combinatorics here can be identified with topology). For example, in representing the convex hull of a set of points, the combinatorics comprises the set of faces (of each dimension) and their incidence relations, and the numerics comprises the point sets determined by the faces, given in some suitable form (say the equations of affine subspace spanned by the faces). There is an implicit consistency relation that must hold between the combinatorics and the numerics. The **consistency approach** to robustness says that our main goal is to ensure that the combinatorics we compute must be part of some consistent geometric object. This amounts to saying that the decisions we make in evaluating predicates during a computation must never be inconsistent. Below we give a slightly more formal version of the consistency approach.

Fortune [27] points out that in principle it is possible to make many algorithms “parsimonious” (meaning that we only perform conditional tests which are independent of the results of previous tests). The basis for this observation is that, assuming all predicates are polynomial sign evaluations, the question of whether the result of current predicate evaluation is a logical consequence of previous evaluations can be phrased as a statement of the existential theory of the reals. This question is at least NP-hard but decidable in polynomial space [17]. Using such a decision procedure, we simply avoid all redundant predicate evaluations.

In practice, of course, we want more than just consistency. Otherwise, we can give the non-redundant predicates any answer we like! In any case, strictly parsimonious algorithms are infeasible. But we can often minimize the dependency between the combinatorial part and numerical part of an algorithm. This is the gist of the Topology-Oriented Approach advocated in a series of papers by Sugihara and Iri [86,85,82,83,81,87]. In making decisions, the combinatorial part is given primacy over the numerical part. As a result, the combinatorial structures is guaranteed to be valid (in the sense of satisfying certain selected properties such as planarity). Somewhat similar ideas are advocated by Schorn [75], but phrased in terms of ensuring properties of primitive operations. For instance, these properties ensure topological properties.

For any particular problem, the topology-oriented approach leaves open as to which topological properties the algorithm designer should pick. A general and systematic theory is possible using the idea of “realizable” combinatorial structure. This is first advocated by Hoffmann, Hopcroft and Karasick [40]. Assume that the algorithm has to compute a geometric object D , which we view as combinatorial structure G together with associated numerical data λ : $D = (G, \lambda)$. Without much loss of generality, we can assume $G = (V, E)$ is a directed graph and $\lambda : V \cup E \rightarrow \mathbb{R}^n$ is a labeling function. Note that this is just a formalized of the “geometry=combinatorics+numerics” formula above; see [93] for further details. While λ may be regarded as arbitrarily specified,

certain choices of λ are defined to be “consistent”. We say G is **valid** if there is a λ which is consistent. In this setting, we can classify an algorithm A to be **geometrically exact** if for any input I , the output $A(I) = (G_I, \lambda_I)$ is such that G_I is the correct structure. For our current purposes, we will not require λ_I to be consistent. We say that A is **consistent** if G_I is valid. Realistically, we would like consistent algorithms to satisfy additional properties: for instance, G_I ought to be the correct structure for some small perturbation I' of I . Hoffmann et al [40] pointed out that designing consistent algorithms may lead to answering hard questions equivalent to theorem proving (akin to Fortune’s parsimonious algorithm).

2.3 Exact Geometric Computation

We now describe an approach that, strictly speaking, ought to be classified under arithmetic approaches. In [94], we call this the **Exact Geometric Computation** (EGC for short) to emphasize that the “exactness” is in the geometry, not in the arithmetic. This is precisely what we defined as geometrically exact in the previous paragraph.

But if EGC is basically an arithmetic approach, how does it ensure anything about geometry? This is a simple but critical point, so we will elaborate on it. As above, assume that our algorithm A on input I computes a geometric object $D_I = (G_I, \lambda_I)$. We had already stated above that *combinatorics (i.e., G_I) is completely determined by the predicate evaluations in the algorithm $A(I)$* . To see this, view each computational step of A as either a **construction step** or a **conditional step**. The former computes new values, and the latter causes the program to branch. Different computational paths lead to different combinatorial outputs. Next, assume that each conditional step evaluates a real predicate and branches depending on the sign of the predicate evaluation $(-1, 0, +1)$. Thus to ensure exact combinatorics, it suffices to ensure that all predicates are evaluated correctly.

In short EGC simply amounts to ensuring that we never err in predicate evaluations. Two important computational consequences flow out of this simple assertion:

(1) *EGC is computationally feasible.* EGC represents a significant relaxation from the naive concept of numerical exactness. We only need to compute to sufficient precision to make the correct predicate evaluation. This has led to development of techniques such as precision-driven computation [96], lazy evaluation [4,3], adaptive computation [80] and floating-point filters [29]. Thus the pessimistic bounds [97] of exact rational arithmetic are unnecessary. Let us expand on this remark, using the example of floating-point filters. Two

preliminary remarks are in order: first, the gold standard of efficiency in scientific computation is machine floating-point arithmetic. Second, this standard works reasonably well for many applications, with non-robustness arising relatively infrequently. Floating-point filters exploit these two remarks, and seek to achieve robustness at the cost of having only a small constant factor overhead over the gold standard. Basically, one maintains some upper bound on the error in the machine floating-point arithmetic. We evaluate predicates using machine arithmetic, using the error bound to determine whether we can trust the result (this is the “filter test”). Only if the filter test fails, we fall back to some other expensive but foolproof evaluation of the same predicate. The cost of this filtered arithmetic has a fixed overhead and a variable overhead: the former is a small constant factor and the latter can be a large overhead. Under the assumption that the filter fails infrequently, the average cost of the variable overhead is also a small constant. In section 4, we treat filters in detail.

(2) *It is possible to create a software library whereby programmers can write robust programs just by calling the library to perform their arithmetic.* In other words, EGC can provide a general and purely arithmetic solution for a large class of problems. By “general solution” we mean that the EGC solution need not¹ be problem-specific or algorithm-specific. This contrasts with the geometric approaches where solutions invented for one problem often do not extend to closely related problems. By “purely arithmetic solution” we mean that the user only has to use the right “number type” in their programs (in place of the standard number types such as `double` found in programming languages such as C or Java). The class of problems which is amenable to such a treatment are the algebraic problems [94]. In the following, we use the term **EGC library** to refer to any software library that supports for exact comparisons.

The algebraic problems constitute the overwhelming majority² of problems treated in contemporary computational geometry. Early studies in EGC assume that the number type is some form of big integer or big rational (e.g., [28]). This suffices for the class of bounded-depth rational problems [94], which already cover all the problems that other approaches have successfully treated. But for a EGC library to reach the algebraic class, we need an additional sophistication not found in big number packages, or computer algebra systems for that matter, that is the ability to guarantee exact comparisons. Such a capability³ was first demonstrated in the `Real/Expr` Package [96]. What

¹ But see [23] for algorithm-specific or [10] for problem-specific EGC analysis.

² The problems found in standard references [71,24,65,8,63,20,33] are all algebraic. In fact it is not easy to find non-algebraic examples, but they arise in special kinds of Voronoi diagrams, in the form of the logarithmic spiral, and in non-holonomic motion planning.

³ Although this was only implemented for $+$, $-$, $\times$, $\div$ and $\sqrt{\cdot}$, the basic technique

Real/Expr offers is the ability to guarantee any number of correct bits when computing a number. These bits can be either relative or absolute bits; in floating-point number representation, relative (resp., absolute) bits means we count the bits from the most significant bit (resp., the radix point). This is generalization of exact comparisons, because the exact comparison of α and β amounts to computing $\alpha - \beta$ to guarantee 1 relative bit. The **Core Library** is a second generation version of **Real/Expr** with two visible changes: it adopts a standard programming semantics for assignments, and provides a natural and simple “numerical accuracy” model (API) [91].

Geometric exactness is a solution to the non-robustness problem because the root cause is numerical errors that lead to wrong predicate evaluations and ultimately inconsistent geometry. By definition, exact geometry is consistent. For many applications, consistent geometry is sufficient. This is especially true in situations where we know the input is approximate anyway. Why bother with *exact* geometry? The reason can be found in a previous remark that ensuring consistency may be as hard as geometry theorem proving [40]. In short, geometric exactness is often the best way to ensure consistency.

EGC is now in the midst of a development that seemed quite remote 10 years ago: it is now possible for any programmer to routinely write completely robust and reasonably efficient geometric code for a large class of important problems. There are basically two⁴ EGC libraries currently: **LEDA** [11,45] and the **Core Library** [48,44]. The **CGAL** library [43] also provides robust algorithms based on EGC principles. The major challenge of EGC is efficiency, both in practical terms as well as theoretically.

At the practical level, users of EGC libraries often find surprising efficiency penalties for innocuous decisions. These typically involve the use of divisions and square-roots in expressions. For instance, in going around the vertices of a polygon $(P_0, P_1, \dots, P_n)$ where $P_0 = P_n$, one might have a while loop conditioned on the test $P_i \neq P_0$ (for $i = 1, 2, \dots, n$) where P_0 is the first point of the polygon. In an actual example, where the P_i ’s are computed points on the unit 2-sphere (with coordinates that is a fraction with a square-root in the denominator), this slowed the computation to a halt. In this case, one should simply perform the check for $i = n$. Another example, if there are common subexpressions involving square-roots, the difference between writing code that shares

extends to all algebraic computation. Moreover, the addition of $\sqrt{\cdot}$ captures the most important subclass of the algebraic problems beyond the rational problems.

⁴ **LEDA** is a very large library that offers many services, including a large suite of data structures and algorithms. We are only referring to a specific facility here, namely the number type called **LEDA_real**. **Core Library** is based on the **Real/Expr Package** [96] which has the basic EGC capability but without the efficient techniques developed in recent years.

these subexpressions and code that does not share can be huge. But this phenomenon should not surprise us – it happens in any high level programming language that is not implemented correctly with the right compiler technology. Thus, all modern compilers are knowledgeable about common subexpressions, loop constants, and a host of other tricks. There is no reason why similar technology should not be developed for EGC to take these concerns out of the minds of programmers. For that matter, using machine floating-point arithmetic is not without its pitfalls for the unwary user. EGC arithmetic is no different.

An area of surprise for users of EGC is numerical input and output. Recall that in EGC, inputs must be assumed to be exact. But if we input a value like 1.23 in conventional languages, this will be converted to the closest machine representation. This can be surprising. In EGC, we represent numbers like $\sqrt{2}$ exactly, but during the computation, some approximation to $\sqrt{2}$ is computed. If we want to output this value to, say, 100 digits, we may not see⁵ this many digits because the currently computed approximation has fewer than 100 correct digits. We refer to [92] for a discussion of many of these issues in the context of the Core Library.

The theoretical challenges of efficiency for EGC libraries relates to high degree algebraic computation. While EGC enables provable robust implementations of a wide range of geometric problems (basically subsuming those problems that other approaches can solve), high degree algebraic computations such as found in CAD applications remain a severe challenge. In this paper, we will address three important areas in which there is significant recent development: constructive root bounds, adaptive precision computation and floating-point filters.

We note two important problem areas, one practical and the other theoretical, that are central to EGC. At present, we have little experience with EGC solutions as embedded in large software systems, such as in a CAD or mesh generation system. Here, we need to cascade the output of one algorithm into the input of another algorithm. The geometric output of an EGC algorithm may be in high precision, and we would like to reduce this precision in the cascade. The **geometric rounding problem** is this: given a consistent geometric object T in high precision, to round it to a consistent object T' at a lower precision. For instance, suppose T is a triangulation (in the plane or higher dimensions). Note that we do not assume that T' is combinatorially equivalent to T – only consistency is required. Otherwise, it is easy to run into NP -hardness, as seen in [60]. In particular, we expect topology to change in T' (e.g., if two close points in T may collapse to a single point in T'). In

⁵ Worst, if the EGC system is incorrectly implemented, we may see 100 digits though many of these may be wrong without the user realizing this.

the robustness literature, “rounding problems” are often a composition of two problems: a construction problem followed by a bona-fide rounding problem. For instance, the problem of snap rounding [39] for intersecting line segments is such a composite problem. In EGC, we prefer to reduce such composite problems to two distinct steps: first compute some geometric object T , and then rounding T to a lower precision version T' . The first step is considered solved using EGC. The second step is what we call the rounding problem. See also [34].

The central theoretical problems relate to the **constant zero problem** for a set Ω of algebraic operators over an appropriate algebraic domain: given a (constant) expression E over Ω , is $E = 0$? The qualification “constant” means that Ω (or E) does not contain any variables. The open questions here relates to non-algebraic expressions. If Ω includes the rational operations and one transcendental function (such as $\sin(x)$ or $\log(x)$), is this problem decidable? The reason for our interest in non-algebraic functions is because, without such a decidability result, we do not even know whether there exists EGC solutions for non-algebraic problems.

3 Constructive Root Bounds

The fundamental decision problem in EGC computation is determining the sign of a constant algebraic expression. Comparison between two algebraic expressions is easily reduced to this problem. Although it is possible to solve this problem by purely algebraic and symbolic means, the current EGC libraries follow a numerical approach first used in `Real/Expr` based on root bounds. The numerical approach seems to be inherently more efficient and depends on the notion of a root bound. Although there are many forms of “root bounds”, our definition below makes it plain that we are interested in bounding roots away from 0.

In this paper, an **expression** E refers to a syntactic object constructed from a given set Ω of operators over the reals $\mathbb{R}$. Each operator in Ω either has a fixed arity (which is a natural number) or is “anadic” (taking any number of arguments). Let $\mathcal{E}(\Omega)$ denote the set of expressions over Ω . For instance, if $\Omega = \{0, 1, x, +, -, \times, \sqrt{\cdot}\}$, then $\mathcal{E}(\Omega)$ is the set of division-free radical expressions over the single variable x . For our purposes, Ω is not allowed to have variables; so each operator of zero arity denotes a real constant. Thus $E \in \mathcal{E}(\Omega)$ denotes a real value $\text{val}(E)$, defined inductively in the natural way. Since some algebraic operators are partial, this value may be undefined (written $\text{val}(E) = \uparrow$). Our expressions are basically straightline programs (cf. [90]), which are basically as rooted, labeled directed acyclic graph (DAG). Thus, sharing of subexpressions is allowed. We will need the sign function, $\text{sign}(E) \in \{-1, 0, +1\}$. But since

$\text{val}(E)$ is partial, the sign function is also partial: $\text{sign}(E) = \uparrow$ iff $\text{val}(E) = \uparrow$. Below, we follow the usual abuse of notation by writing “ E ” instead of $\text{val}(E)$.

Definition 1 We call $b > 0$ a **root bound** for an expression E if the following holds: if $E \neq \uparrow$ and $E \neq 0$ then $|E| \geq b$. We also say $(-\log_2 b)$ is a **root bit-bound** for E .

To determine the sign of E from a root bound, we compute a numerical approximation $\tilde{E}$ such that if $E = \uparrow$ then $\tilde{E} = \uparrow$; otherwise, $|E - \tilde{E}| < \frac{b}{2}$. Then

$$\text{sign}(E) = \begin{cases} \text{sign}(\tilde{E}) & \text{if } |\tilde{E}| \geq \frac{b}{2} \text{ or } \tilde{E} = \uparrow \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

Following [52], we classify the algebraic expressions into five categories as follows:

- $\Omega_0 = \{\pm, \times\} \cup \mathbb{Z}$ (where $\mathbb{Z}$ are the integers). Thus Ω_0 -expressions are polynomials.
- $\Omega_1 = \Omega_0 \cup \{\div\}$. Thus Ω_1 -expressions are rational expressions.
- $\Omega_2 = \Omega_1 \cup \{\sqrt[n]{\cdot} : n \geq 2\}$. Thus Ω_2 -expressions are radical expressions.
- $\Omega_3 = \Omega_2 \cup \{\text{Root}(P, i) : P \in \mathbb{Z}[x]\}$. Here $\text{Root}(P, i)$ means the i -th real root of the polynomial P , which is assumed to be presented by its sequence of $n + 1$ integer coefficients if $\deg(P) = n$.
- $\Omega_4 = \Omega_2 \cup \{\text{Root}(P, i) : P \in \Omega_4[x]\}$. Thus Ω_4 expressions include general algebraic expressions.

We can see from (1) that root bounds determine the worst-case complexity in exact sign determination. Thus, a main problem here is to find tight root bounds, and the efficient ways to compute them.

3.1 Review of Constructive Root Bounds

The problem of root bounds has been extensively studied (e.g., [55] or [59, chap. 2]). However, from an algorithmic point of view, many classical results are not constructive. Here the notion of “constructive” depends on the representation of expressions. Directed acyclic graphs (DAGs) are common forms of representation. In such cases, constructive bounds could mean those that can be computed inductively on the DAG structure. Thus, the constructive root bound problem is stated as: given a set $\mathcal{E}$ of expressions (e.g., the radical expressions), give a set of inductive rules for computing a root bound for each expression in $\mathcal{E}$. Typically, the inductive rules are based on some properties about the generation polynomials (of which the expression concerned is

E	d	ℓ	h
rational $\frac{a}{b}$	1	$\sqrt{a^2 + b^2}$	$\max\{ a , b \}$
$E_1 \pm E_2$	$d_1 d_2$	$\ell_1^{d_2} \ell_2^{d_1} 2^{d_1 d_2 + \min\{d_1, d_2\}}$	$(h_1 2^{1+d_1})^{d_2} (h_2 \sqrt{1+d_2})^{d_1}$
$E_1 \times E_2$	$d_1 d_2$	$\ell_1^{d_2} \ell_2^{d_1}$	$(h_1 \sqrt{1+d_1})^{d_2} (h_2 \sqrt{1+d_2})^{d_1}$
$E_1 \div E_2$	$d_1 d_2$	$\ell_1^{d_2} \ell_2^{d_1}$	$(h_1 \sqrt{1+d_1})^{d_2} (h_2 \sqrt{1+d_2})^{d_1}$
$\sqrt[k]{E_1}$	$k d_1$	ℓ_1	h_1

Table 1

Rules for degree-length and degree-height bounds

a root). A number of constructive root bounds have been proposed.

Canny’s bound. Canny [16] shows that given a zero-dimensional system Σ of n polynomial equations with n unknowns, if $(\alpha_1, \dots, \alpha_n)$ is a solution, then $|\alpha_i| \geq (3dc)^{-nd^n}$ for all non-zero component α_i . Here c (resp., d) is an upper bound on the absolute value of coefficients (resp., the degree) of any polynomial in the system. An important proviso in Canny’s bound is that the homogenized system $\hat{\Sigma}$ has a non-vanishing U -resultant. Equivalently, $\hat{\Sigma}$ has finitely many roots at infinity. Yap [95, p. 350] gives the treatment for the general case, based on the notion of “generalized U -resultant”. Such multivariate root bounds are easily translated into a bound on expressions, as discussed in [12].

Degree-length and degree-height bounds. The degree-length bound [95] is a bound for general algebraic expressions, based on Landau’s root bound. For an expression E , the algorithm computes the upper bounds on the degree d and on length ($\|\cdot\|_2$) ℓ of the minimal polynomial of E . If $E \neq 0$, then from Landau’s bound we know $|E| \geq \frac{1}{\ell}$. The extended Hadamard bound on polynomial matrix [32] is used to compute an upper bound of ℓ . A similar degree-height bound based on Cauchy’s root bound is found in [96]. Here “length” and “height” refer to the 2-norm and ∞ -norm of a polynomial, respectively. Both results are based on the resultant calculus. The bounds are maintained inductively on the structure of the expression DAG using the recursive rules found in Table 1, where the parameters d, ℓ, h denote the upper bounds for the degree, length and height, respectively.

Degree-measure bound. It is known that if $\alpha \neq 0$, we have

$$\frac{1}{m(\alpha)} \leq |\alpha| \leq m(\alpha). \quad (2)$$

	E	$u(E)$	$l(E)$
1.	integer a	$ a $	1
2.	$E_1 \pm E_2$	$u(E_1)l(E_2) + l(E_1)u(E_2)$	$l(E_1)l(E_2)$
3.	$E_1 \times E_2$	$u(E_1)u(E_2)$	$l(E_1)l(E_2)$
4.	$E_1 \div E_2$	$u(E_1)l(E_2)$	$l(E_1)u(E_2)$
5.	$\sqrt[k]{E_1}$	$\sqrt[k]{u(E_1)}$	$\sqrt[k]{l(E_1)}$

Table 2

BFMS Rules

Here, the measure $m(\alpha)$ is defined as $|a_m| \cdot \prod_{i=1}^m \max\{1, |\alpha_i|\}$, where a_m is the leading coefficient of α 's minimal polynomial, and α_i 's are the conjugates of α .

Let α and β be two nonzero algebraic numbers of degrees m and n respectively. The following relations on measures are given in [59],

$$\begin{aligned}
m(\alpha \pm \beta) &\leq 2^{mn} m(\alpha)^n m(\beta)^m \\
m(\alpha \times \beta) &\leq m(\alpha)^n m(\beta)^m \\
m(\alpha \div \beta) &\leq m(\alpha)^n m(\beta)^m \\
m(\alpha^{1/k}) &\leq m(\alpha) \\
m(\alpha^k) &\leq m(\alpha)^k
\end{aligned}$$

Based on Mignotte's work, Burnikel et al [12] develop recursive rules to maintain the upper bounds for degrees and measures of radical expressions and call it the *degree-measure bound*. The degree-measure bound turns out to be always better than the degree-length bound [12]. Improvements over the original degree-measure bound are reported in [52,78].

BFMS bound. One of the best constructive root bounds for the class of radical expressions is from Burnikel et al [12] (hereafter called the “BFMS bound”). For division-free expressions, it is an improvement over previously known bounds and is essentially tight. But in presence of divisions, the BFMS bound is not necessarily an improvement of the degree-measure bound. Conceptually the BFMS approach first transforms a radical expression E to a quotient of two division-free expressions $U(E)$ and $L(E)$. Two parameters $u(E)$ and $l(E)$, the upper bounds on the conjugates of $U(E)$ and $L(E)$, respectively, are maintained by the recursive rules in Table 2. Clearly, if E is division-free, then $L(E) = 1$ and $\text{val}(E)$ is an algebraic integer (i.e., a root of some monic integer polynomial).

For an expression E having r radical nodes with indices $k_1, k_2, \dots, k_r$, the

BFMS bound is given by

$$\text{val}(E) \neq 0 \Rightarrow (u(E)^{D(E)^2-1}l(E))^{-1} \leq |\text{val}(E)| \leq u(E)l(E)^{D(E)^2-1}, \quad (3)$$

where $D(E) = \prod_{i=1}^r k_i$, and $u(E)$ and $l(E)$ are (respectively) upper bounds on the absolute values of algebraic conjugates of $\text{val}(U(E))$ and $\text{val}(L(E))$.

For division-free expressions, the BFMS bound improves to

$$\text{val}(E) \neq 0 \Rightarrow |\text{val}(E)| \geq (u(E)^{D(E)-1})^{-1}. \quad (4)$$

Improved and generalized BFMS bound. Note that the root bit-bound in (3) is quadratic in $D(E)$, while in (4), it is linear in $D(E)$. This quadratic factor can be a serious efficiency issue. Consider a simple example: $E = (\sqrt{x} + \sqrt{y}) - \sqrt{x + y + 2\sqrt{xy}}$ where x, y are L -bit integers. Of course, this expression is identically 0 for any x, y . The BFMS bound yields a root bit-bound of $7.5L + \mathcal{O}(1)$ bits. But in case, x and y are viewed as rational numbers (with denominator 1), the bit-bound becomes $127.5L + \mathcal{O}(1)$. This example shows that introducing rational numbers at the leaves of expressions has a major impact on the original BFMS bound.

Recently, Mehlhorn et al [57] have extended the BFMS bound to support general algebraic expressions (the Ω_4 expressions). Their new root bit-bound depends on $D(E)$ linearly for most algebraic expressions. The significant improvement is obtained from a trick to avoid the doubling of the degree in transforming E into a division of two algebraic integer expressions. For radical expressions, it can be shown that this new bound is always an improvement over the original one, and depends on $D(E)$ linearly.

Scheinerman bound. This bound adopts an interesting approach based on matrix eigenvalues [73]. Let $\Lambda(n, b)$ denote the set of eigenvalues of $n \times n$ matrices with integer entries with absolute value at most b . It is easy to see that $\Lambda(n, b)$ is a finite set of algebraic integers. Moreover, if $\alpha \in \Lambda(n, b)$ is non-zero then $|\alpha| \geq (nb)^{1/n}$. Scheinerman gives a constructive root bound for division-free radical expressions E by maintaining two parameters, $n(E)$ and $b(E)$, satisfying the property that the value of E is in $\Lambda(n(E), b(E))$. These recursive rules are given by Table 3.

Note that the rule for $\sqrt{cd}$ is rather special, but it can be extremely useful. In Rule 6, the polynomial $\bar{P}(x)$ is given by $\sum_{i=0}^d |a_i| x^i$ when $P(x) = \sum_{i=0}^d a_i x^i$. This rule is not explicitly stated in [73], but can be deduced from an example he gave. An example given in [73] is to test whether $\alpha = \sqrt{2} + \sqrt{5 - 2\sqrt{6}} - \sqrt{3}$

	E	$n(E)$	$b(E)$
1.	integer a	1	$ a $
2.	$\sqrt{cd}$	2	$\max\{ c , d \}$
3.	$E_1 \pm E_2$	$n_1 n_2$	$b_1 + b_2$
4.	$E_1 \times E_2$	$n_1 n_2$	$b_1 b_2$
5.	$\sqrt[k]{E_1}$	kn_1	b_1
6.	$P(E_1)$	n_1	$\overline{P}(n_1 b_1)$

Table 3

Scheinerman's Rules

is zero. Scheinerman's bound requires calculating α to 39 digits while the BFMS bound says 12 digits are enough.

Li-Yap bound. In [52], we give a new constructive root bound that is applicable to a general class of algebraic expressions (Ω_3). Our basic idea is to bound the leading and tail coefficients, and the conjugates of the algebraic expression with the help of resultant calculus. The new bound gives significantly better performance in many important computations involving divisions and root extractions. For any algebraic number α , we will exploit the following relation:

$$\alpha \neq 0 \Rightarrow |\alpha| \geq (\mu(\alpha)^{\deg(\alpha)-1} \text{lead}(\alpha))^{-1}, \quad (5)$$

where $\mu(\alpha) = \max\{|\xi| : \xi \text{ is a conjugate of } \alpha\}$, $\deg(\alpha)$ is the degree of the minimal polynomial $\text{Irr}(\alpha)$ of α and $\text{lead}(\alpha)$ is the leading coefficient of $\text{Irr}(\alpha)$. A similar relation was also used in [12] for bounds on algebraic integers. Our bound requires the computation of three upper bounds

$$D(E), \quad \text{lc}(E), \quad \overline{\mu}(E)$$

on the corresponding parameters $\deg(E)$, $\text{lead}(E)$ and $\mu(E)$. Suppose that E has k radical nodes or root-of-polynomial nodes $\{r_1, r_2, \dots, r_k\}$. We choose $D(E) = \prod_{i=1}^k k_i$ where k_i is either the index of r_i if r_i is a radical node, or the degree of the polynomial if r_i is a polynomial-root node. Due to the admission of division, we also need to maintain upper bounds $\text{tc}(E)$, $M(E)$ on $\text{tail}(E)$ and $m(E)$ in bounding $\text{lead}(E)$. Here the tail coefficient $\text{tail}(E)$ is defined as the constant term of $\text{Irr}(E)$.

Table 4 gives the recursive rules to maintain $\text{lc}(E)$, $\text{tc}(E)$ and $M(E)$.

The upper bounds on conjugates, $\overline{\mu}(E)$, are obtained through resultant calculus and standard interval arithmetic techniques. It turns out that it is nec-

	E	$\text{lc}(E)$	$\text{tc}(E)$	$M(E)$
1.	rational $\frac{a}{b}$	$ b $	$ a $	$\max\{ a , b \}$
2.	$\text{Root}(P)$	$ \text{lead}(P) $	$ \text{tail}(P) $	$\ P\ _2$
3.	$E_1 \pm E_2$	$\text{lc}_1^{D_2} \text{lc}_2^{D_1}$	$M_1^{D_2} M_2^{D_1} 2^{D(E)}$	$M_1^{D_2} M_2^{D_1} 2^{D(E)}$
4.	$E_1 \times E_2$	$\text{lc}_1^{D_2} \text{lc}_2^{D_1}$	$\text{tc}_1^{D_2} \text{tc}_2^{D_1}$	$M_1^{D_2} M_2^{D_1}$
5.	$E_1 \div E_2$	$\text{lc}_1^{D_2} \text{tc}_2^{D_1}$	$\text{tc}_1^{D_2} \text{lc}_2^{D_1}$	$M_1^{D_2} M_2^{D_1}$
6.	$\sqrt[k]{E_1}$	lc_1	tc_1	M_1
7.	E_1^k	lc_1^k	tc_1^k	M_1^k

Table 4

Recursive rules for $\text{lc}(E)$ (and associated $\text{tc}(E)$ and $M(E)$)

essary to maintain a lower bound $\underline{\nu}(E)$ on the conjugates at the same time. The recursive rules to maintain these two bounds are given in Table 5.

	E	$\overline{\mu}(E)$	$\underline{\nu}(E)$
1.	rational $\frac{a}{b}$	$ \frac{a}{b} $	$ \frac{a}{b} $
2.	$\text{Root}(P)$	$1 + \ P\ _\infty$	$(1 + \ P\ _\infty)^{-1}$
3.	$E_1 \pm E_2$	$\overline{\mu}(E_1) + \overline{\mu}(E_2)$	$\max\{M(E)^{-1}, (\overline{\mu}(E)^{D(E)-1} \text{lc}(E))^{-1}\}$
4.	$E_1 \times E_2$	$\overline{\mu}(E_1) \overline{\mu}(E_2)$	$\underline{\nu}(E_1) \underline{\nu}(E_2)$
5.	$E_1 \div E_2$	$\overline{\mu}(E_1) / \underline{\nu}(E_2)$	$\underline{\nu}(E_1) / \overline{\mu}(E_2)$
6.	$\sqrt[k]{E_1}$	$\sqrt[k]{\overline{\mu}(E_1)}$	$\sqrt[k]{\underline{\nu}(E_1)}$
7.	E_1^k	$\overline{\mu}(E_1)^k$	$\underline{\nu}(E_1)^k$

Table 5

Recursive rules for bounds on conjugates

Finally, we obtain the new root bound as follows: Given an Ω_3 -expression E , if $E \neq 0$, then

$$|E| \geq (\overline{\mu}(E)^{(D(E)-1)} \text{lc}(E))^{-1}. \quad (6)$$

We implemented the new bound in our Core Library and experiments show that it can achieve remarkable speedup over previous bounds in the presence of division [52]. Although we have described our bounds for the class of Ω_3 -expressions, it should be clear that our method extends to more general expressions.

Comparisons between various constructive root bounds can be found in [12,52]. In general, a direct comparison of the above root bounds is a difficult task because of the different choice of parameters and bounding functions used. Therefore, following the tact in [52], we compare their performance on various special subclasses of algebraic expressions.

1. For division-free radical expressions, the BFMS bound is never worse than all the other bounds. Moreover, for this special class of expressions, Li-Yap bound is identical to the BFMS bound.
2. For general algebraic expressions, in terms of root bit-bound, Li-Yap bound is at most $D \cdot M$ where D is the degree bound, and M is the root bit-bound from the degree-measure bound.
3. Considering the sum of square roots of rational numbers (a common problem in solving the shortest Euclidean path problem), it can be shown that each of Li-Yap bound and the degree-measure bound can be better than the other depending on different parameters about the expressions. But both of them are always better than the BFMS bound.
4. Given a radical expression E with rational values at the leaves, if E has no divisions and shared radical nodes, Li-Yap bound for E is never worse than the BFMS bound, and can be better in many cases.
5. A critical test in Fortune's sweepline algorithm is to determine the sign of the expression $E = \frac{a+\sqrt{b}}{d} - \frac{a'+\sqrt{b'}}{d'}$ where a 's, b 's and d 's are $3L$ -, $6L$ - and $2L$ -bit integers, respectively. The BFMS bound requires $(79L + 30)$ bits and the degree-measure (D-M) bound needs $(64L + 12)$ bits. Li-Yap root bit-bound improves them to $(19L + 9)$ bits. We generate some random inputs with different L values which always make $E = 0$, and put the timings (in seconds) of the tests in Table 6. The experiments are performed on a Sun UltraSPARC with a 440 MHz CPU and 512MB main memory.

L	10	20	50	100	200
NEW	0.01	0.03	0.12	0.69	3.90
BFMS	0.03	0.24	1.63	11.69	79.43
D-M	0.03	0.22	1.62	10.99	84.54

Table 6
Timings for Fortune's expression

Root separation bounds. In both the Core Library and LEDA, the comparison of two expressions α and β is obtained by computing the root bound of $\alpha - \beta$. However, we can use root separation bounds [95] for any polynomial P that has α and β as roots. If $P(X) \in \mathbb{C}[X]$ is a non-zero polynomial, $\text{sep}(P)$ denotes the minimum $|\alpha_i - \alpha_j|$ where $\alpha_i \neq \alpha_j$ range over all pairs of complex roots of P . When P has less than two distinct roots, define $\text{sep}(P) = \infty$. The following general bound of Rump [72] (as rectified by Schwartz [76]) is

$$\text{sep}(P) > \left[2 \cdot m^{m/2+2} (\|P\|_\infty + 1)^m \right]^{-1}$$

where m is the degree of P . Suppose $A(X)$ and $B(X)$ are the minimal polynomials for α and β , then $|\alpha - \beta| \geq \text{sep}(AB)$. Therefore, if we maintain upper bounds d, d' on the degrees of A and B , and upper bounds h, h' on the heights of A and B , we obtain

$$|\alpha - \beta| \geq [hh'(1+n)]^{-2n+1} (2n)^{-n-1} \quad (7)$$

where $n = \max\{d, d'\}$ (see [95, p.173]). The advantage of using equation (7) is that the root bit bound does not have a dd' term, as would be the case if we use resultant calculus. Note that using this bound does not nicely fit into our recursive root bound framework (in particular, it does not generate bounds for a new minimal polynomial). On the other hand, it is unnecessary to maintain parameters for the minimal polynomial of $\alpha - \beta$ since this is not an actual expression that the user constructed.

Zero test. Zero testing is the special case of sign determination in which we want to know whether an expression is zero or not. Many predicates in computational geometry programs are really zero tests (e.g. detection of degeneracy, checking if a point lies on a hyperplane). In other applications, even though we need general sign determination, the zero outcome is very common. For instance, in the application of EGC to theorem proving [90], true conjectures are equivalent to the zero outcome. In our numerical approach based on root bounds, the complexity of sign determination is determined by the root bound when the outcome is zero. Since root bounds can be overly pessimistic, such tests can be extremely slow. Hence it is desirable to have an independent method of testing if an expression is zero. Such a zero test can be used as a filter for the sign determination algorithm. Only when the filter detects a non-zero do we call the iterative precision numerical method.

Yap and Blömer [6] noted that for the case of sum of square roots of integers,

zero testing is deterministic polynomial time while the sign determination problem is not known to be polynomial time. Blömer [5] gave a probabilistic algorithm for un-nested real radical expressions. When the radicals are nested, we can apply denesting algorithms [46,50]. Note that these methods are non-numerical methods.

4 Precision-Driven Computation

The idea of adaptive precision computation has many manifestations in EGC. Here, we want to describe the approach called **precision-driven computation** [96], i.e. we wish to approximate the value of an expression E to some user-specified precision p . We will propagate the precision p to all the nodes in E using suitable inductive rules. In the simplest case, this propagation may proceed only in one direction from the root down to the leaves. Then, we evaluate the approximate values at the leaves and recursively apply the operations at each node from the bottom up. In our implementations, the approximate values at each node is a big float number. Complications arise when the propagation of precision requires bounds on the magnitude of the values at some nodes. This is where the root bounds from the previous section become essential.

In the literature, a technique called “lazy evaluation” (e.g. [4]) has superficial similarities to our method. However, lazy evaluation typically “pumps” increasingly precise values from the leaves to the root, and simply tracks the forward error. If this error is larger than the desired precision at the root, the process is iterated. Note that this is insufficient to guarantee the sign of an expression (in particular, it cannot detect a zero value in a finite number of iterations).

In the following, we will describe some improved methods for precision-driven evaluation of radical expressions (usually represented as DAGs internally). It is assumed that the leaves of these expressions are rational constants.

4.1 Guaranteed Precision Approximation

We use a notion of numerical precision from [96] which combines both absolute and relative precisions in one framework: given a real number X , and extended reals $a, r \in \mathbb{R} \cup \pm\infty$, we say that a real number $\widetilde{X}$ is an *approximation* of another real X to *(composite) precision* $[r, a]$, denoted $\widetilde{X} \simeq X[r, a]$, provided either

$$|\widetilde{X} - X| \leq 2^{-r} |X| \quad \text{or} \quad |\widetilde{X} - X| \leq 2^{-a}.$$

When $a = +\infty$ (resp., $r = +\infty$), then $[r, a]$ is simply a relative (resp., absolute) precision bound. In practice, r and a will be integers. Note that under the above definition, when $X = 0$, the relative precision condition actually requires an exact evaluation.

As we discussed in Section 3, expressions are usually constructed as a DAG. Along with this construction, we first compute the root bounds from bottom-up. The next step is to approximate the expression to some specified absolute or relative precision. For example, a special case, sign determination, can be solved by approximating the expression to an absolute precision determined by root bounds.

In the precision driven approach, the approximation process starts with propagating precision requirements down the DAG. An algorithm to propagate composite precision bounds is described in [67, chapter 7]. Here, we describe a simpler and more accurate method. First we translate the composite bound $[r, a]$ to an **absolute error bound** α_E such that if $|X - \tilde{X}| \leq \alpha_E$ then $X \simeq X[r, a]$. It is not hard to see that it is sufficient to set

$$\alpha_E = \max\{2^{-r} |E|, 2^{-a}\}.$$

Note that although here we may use the value $|E|$ (actually only needed when r is finite), in practice, we use its lower bound to avoid the exact evaluation of E . We will discuss how to compute such bounds in the following sections.

The rules to propagate α_E to the children of E are presented in the second column of Table 7. If E_1, E_2 are the children of E , we want to define α_{E_i} ($i = 1, 2$) in such a way that if $\tilde{E}_i$ satisfies $|E_i - \tilde{E}_i| \leq \alpha_{E_i}$ then $\tilde{E}$ satisfies $|E - \tilde{E}| \leq \alpha_E$. For simplicity, we write α_{E_i} for α_i . Here, $\tilde{E}$ is obtained by applying the operator at E to $\tilde{E}_1, \tilde{E}_2$, computed to some specified precision. The rule for computing this approximation is given in column 3 of Table 7. A notation used in column 3 is that, for any real X and $\alpha > 0$, $(X)_\alpha$ refers to any approximation $\tilde{X}$ for X that satisfies the bound $|X - \tilde{X}| \leq \alpha$.

In summary, column 2 tells us how to propagate absolute precision bounds downward towards the leaves, and column 3 tells us how to compute approximations from the leaves upward to the root. At the leaves, we assume an ability to generate numerical approximations to the within the desired error bounds. At each node F , our rules guarantee that the approximate value at F satisfies the required absolute precision bound α_F .

Note that for the addition, subtraction and multiplication operations, the computation of $\tilde{E}$ can be performed exactly (as in [51]). But the present rules no longer require exact computation. The new rule is clearly never worse than the old rule (at the cost of at most one extra bit), but is sensitive to the actual precision needed. In fact, for all operations, we now allow an absolute error

of $\frac{\alpha_E}{2}$. Let us briefly justify the rule for $E_1 \times E_2$ in Table 7. It is sufficient to ensure that $|E - \widetilde{E}_1 \widetilde{E}_2| \leq \alpha_E/2$. But $|E - \widetilde{E}_1 \widetilde{E}_2| \leq \alpha_1 |E_2| + \alpha_2 |E_1| + \alpha_1 \alpha_2 \leq \frac{\alpha_E}{c} + \frac{\alpha_E}{c} + \frac{\alpha_E^2}{c^2}$. So it is sufficient to ensure that $\frac{\alpha_E}{c} + \frac{\alpha_E}{c} + \frac{\alpha_E^2}{c^2} \leq \alpha_E/2$. Solving for c , we obtain $c \geq 2 + \sqrt{4 + 2\alpha_E}$. See [51] for justifications of the other entries.

E	Downward Rules	Upward Rules
$E_1 \pm E_2$	$\alpha_1 = \alpha_2 = \frac{1}{4}\alpha_E$	$\widetilde{E} = (\widetilde{E}_1 \pm \widetilde{E}_2)_{\frac{\alpha_E}{2}}$
$E_1 \times E_2$	If $\alpha_E \geq E $ then return to parent node with $\widetilde{E} = 0$. else let $c \geq 2 + \sqrt{4 + 2\alpha_E}$, and $\alpha_1 = \frac{\alpha_E}{c} \min\{1, 1/ E_2 \}$, $\alpha_2 = \frac{\alpha_E}{c} \min\{1, 1/ E_1 \}$.	$\widetilde{E} = (\widetilde{E}_1 \times \widetilde{E}_2)_{\frac{\alpha_E}{2}}$
$E_1 \div E_2$	$\alpha_1 = \alpha_E E_2 /4$, $\alpha_2 = \frac{\alpha_E E_2 }{4 E + 2\alpha_E}$	$\widetilde{E} = (\widetilde{E}_1 \oslash \widetilde{E}_2)_{\frac{\alpha_E}{2}}$
$\sqrt[k]{E_1}$	$\alpha_1 = \alpha_E \sqrt[k]{E_1}^{k-1}/2$	$\widetilde{E} = (\sqrt[k]{\widetilde{E}_1})_{\frac{\alpha_E}{2}}$

Table 7

Rules for (1) Propagating absolute precision α_E and (2) Approximation $\widetilde{E}$

Actually, column 2 is not exactly the rule one uses in implementation. We give formulas for α_i ($i = 1, 2$) in terms of $|E_1|$ and $|E_2|$ to make the formulas easier to understand. But it is generally not possible nor desirable to compute $|E_i|$ exactly. Instead, we compute tight upper and lower bounds on the logarithms of the expressions given for α_1 and α_2 , respectively.

It is obvious that if the α_E at the root is 0, the rules in Table 7 basically say that it requires an exact evaluation for each underlying node. The system may reject some of such numerical approximation requests because that cannot always be finished within finite time and space, particularly when the expression is nonrational.

4.2 Bounds on the Magnitude of Expressions

In **Real/Expr** and **Core Library**, the evaluation begins by computing bounds on the absolute value of each node (see [67]). Such bounds are needed for two purposes: 1) propagating absolute precision bounds using the rules in Table 7, and 2) translating a finite relative precision bound at root into an equivalent absolute one.

We now review this magnitude bounds computation. For any expression E , we define $\text{MSB}(E)$ to be $\lfloor \lg(|E|) \rfloor$. Intuitively, the MSB of E tells us about the location of the most significant bit of E . By definition, the $\text{MSB}(0) = -\infty$. For efficiency purpose in practice, we will compute a bounding interval

$[\mu_E^-, \mu_E^+]$ that contains $\text{MSB}(E)$, instead of computing its true value. The rules in Table 8 are used to maintain this interval.

E	μ_E^+	μ_E^-
rational $\frac{a}{b}$	$\lceil \lg(\frac{a}{b}) \rceil$	$\lfloor \lg(\frac{a}{b}) \rfloor$
$E_1 \pm E_2$	$\max\{\mu_{E_1}^+, \mu_{E_2}^+\} + 1$	$\lfloor \lg(E) \rfloor$
$E_1 \times E_2$	$\mu_{E_1}^+ + \mu_{E_2}^+$	$\mu_{E_1}^- + \mu_{E_2}^-$
$E_1 \div E_2$	$\mu_{E_1}^+ - \mu_{E_2}^-$	$\mu_{E_1}^- - \mu_{E_2}^+$
$\sqrt[k]{E_1}$	$\lceil \mu_{E_1}^+ / k \rceil$	$\lfloor \mu_{E_1}^- / k \rfloor$

Table 8

Rules for upper and lower bounds on $\text{MSB}(E)$

The main subtlety in this table is the entry for μ_E^- when $E = E_1 \pm E_2$. We call this the **special entry** of this table because, due to potential cancellation, we cannot derive a lower bound on $\text{MSB}(E)$ in terms of the bounds on E_1 and E_2 only. If the MSB bounds cannot be obtained from the fast floating-point filtering techniques, there are two possible ways to determine this entry in practice. First, we could approximate E numerically to obtain its most significant bit, or to reach the root bound in case $E = 0$. Thus, this method really determines the true value of $\text{MSB}(E)$, and provides the entry shown in the above table. This numerical approximation process can be conducted in a progressive and adaptive way (e.g., doubling the absolute precision in each iteration, until it finds out the exact MSB, or reaches the root bit-bound). The second method is applicable only under certain conditions: either when E_1 and E_2 have the same (resp., opposite) sign in the addition (resp., subtraction) case, or their magnitudes are quite different (by looking at their MSB's). In either case, we can deduce the μ_E^- from the bounds on E 's children. For instance, if $\mu_{E_1}^- > \mu_{E_2}^+ + 1$ then $\mu_{E_1}^- - 1$ is a lower bound for μ_E^- . This approach could give better performance if the signs of both operands are relatively easier to be obtained.

Remark: In the implementation, one often assumes that μ_E^- and μ_E^+ are machine integers. But it may be better to allow these to be machine floats, since this can yield sharper bounds. Furthermore, we can re-interpret μ_E^- and μ_E^+ to be upper and lower bounds on $\lg |E|$ (and not $\lfloor \lg |E| \rfloor$). Note that $|\mu_E^+ - \mu_E^-| \leq 2^{m+1}$ where m is the number of operations in E .

4.3 The Approximation and MSB Algorithms

There are two important algorithms which we derive from the above tables: one is $\text{APPROX}(E, \alpha_E)$ which computes an approximation of E to within the absolute precision α_E . The other algorithm is $\text{ComputeMSB}(E, \text{needUMSB})$,

needLMSB), which computes upper and/or lower bounds for $\text{MSB}(E)$, following the rules in Table 8. Another algorithm of interest is the sign determination, which can be reduced to an approximation to appropriate root bounds.

The algorithms APPROX and ComputeMSB form the basis of guaranteed precision computation in our Core Library. It should be noted that APPROX and ComputeMSB are mutually recursive algorithms: this is because ComputeMSB will need to call APPROX to compute the special entry (for $\mu_{E_1 \pm E_2}^-$) in Table 8. Clearly, APPROX needs ComputeMSB for the downward rules (except for addition or subtraction) in Table 7. It is not hard to verify that this mutual recursion will not lead to infinite loops based on two facts: 1) the underlying graph of E is a DAG, and 2) for the addition/subtraction node, the ComputeMSB may need to call APPROX, while the APPROX will not call ComputeMSB at the same node again.

APPROX The APPROX algorithm has two steps: 1) distributing the precision requirement down the DAG, and 2) calculating an approximate value from leaves up to the root. Here we only explain step (1) in details. We refer the reader to [67] for details on the implementation of multi-precision arithmetic in our libraries.

By looking at Table 7, we see that, for the purpose of precision propagation, we do not have to compute the MSB bounds for all the nodes in an expression DAG. Instead, one can deduce the following from the rules in that table:

- Addition and subtraction $E = E_1 \pm E_2$. No MSB bounds on E_1, E_2 are needed to propagate precision bounds.
- Multiplication $E = E_1 \times E_2$. Only the μ^+ 's of E_1, E_2 are needed.
- Division $E = E_1 \div E_2$. Only the bounds $\mu_{E_1}^+$ and $\mu_{E_2}^-$ are needed.
- Root extraction $E = \sqrt[k]{E_1}$. Only $\mu_{E_1}^-$ is needed.

ComputeMSB Before precision requirements can be distributed based on Table 7, the necessary MSB bounds, as discussed above, must be first computed. We now develop a more sophisticated ComputeMSB algorithm to compute only the required upper and lower MSB bounds. Let the refined algorithm take two additional arguments: $\text{ComputeMSB}(E, \text{needUMSB}, \text{needLMSB})$ where needUMSB and needLMSB are Boolean flags. The needUMSB (resp., needLMSB) flag says that an upper (resp., lower) bound on $\text{MSB}(E)$ is needed.

The lower and upper MSB bounds are needed in three places:

- (1) in propagating absolute precisions from the root of the expression DAG;

- (2) in the “relative-to-absolute” precision conversion at the root, when a finite relative precision requirement r is present. Note that this is not an issue in sign determination, in which we only use absolute precision;
- (3) in the inductive rules to compute MSB bounds from (see Table 8).

Remember that APPROX is called in computing exact MSB’s for the “special entry” in Table 7. However, this does not require extra MSB bounds to be computed, since all bounds for distributing precisions have already been covered by the above item (1). Although all the MSB bounds can be computed in a single top-down traversal of the DAG, for clarity, we compute them through the mutual recursion of the ComputeMSB and APPROX algorithms. In the algorithm, we prevent repeated computation of each bound by first checking whether it has been computed when visiting a node. We simply present a self-explanatory ComputeMSB algorithm here in a C++-like syntax:

Algorithm

```

ComputeMSB(E, needUMSB, needLMSB) {
    if (E.umsb was computed) needUMSB = false;
    if (E.lmsb was computed) needLMSB = false;
    if (both needUMSB and needLMSB are false) return;

    switch (E.operation_type) {
    case 'constant':
        if (needUMSB) E.umsb = ceilLog(E.value);
                                // ceilLog is ceiling of log_2
        if (needLMSB) E.lmsb = floorLog(E.value);
                                // floorLog is floor of log_2
        break;
    case '+' or '-':
        if (needUMSB and (not needLMSB)) {
            ComputeMSB(E.first, true, false);
            ComputeMSB(E.second, true, false);
            E.umsb = max{E.first.umsb, E.second.umsb} + 1;
        }
        if (needLMSB) {
            APPROX(E, E.root_bound);
            E.umsb = ceilLog(E.value);
            E.lmsb = floorLog(E.value);
        }
        break;
    case '*':
        ComputeMSB(E.first, needUMSB, needLMSB);
        ComputeMSB(E.second, needUMSB, needLMSB);
        if (needUMSB) E.umsb = E.first.umsb + E.second.umsb;
        if (needLMSB) E.lmsb = E.first.lmsb + E.second.lmsb;
    }
}

```

```

        break;
    case '/':
        ComputeMSB(E.first, needUMSB, needLMSB);
        ComputeMSB(E.second, needLMSB, needUMSB);
        if (needUMSB) E.umsb = E.first.umsb - E.second.lmsb;
        if (needLMSB) E.lmsb = E.first.lmsb - E.second.umsb;
        break;
    case 'k-th root extraction':
        ComputeMSB(E.first, needUMSB, needLMSB);
        if (needUMSB) E.umsb = E.first.umsb / k;
        if (needLMSB) E.lmsb = E.first.lmsb / k;
        break;
} //switch
} //ComputeMSB

```

5 Numerical Filters and Certification

In the EGC techniques of the previous sections, the use of multi-precision arithmetic is essential. Another avenue to gain efficiency is to exploit machine floating-point arithmetic which is fast and highly optimized on current hardware. The basic idea is simple: we must “check” or “certify” the output of machine evaluation of predicates, and only go for the slower exact methods when this fails.

Model for checking and filtering. We give a simple formal model for checking and filtering (certifying). Assume I and O are some sets called the **input and output spaces**. In numerical problems, it is often possible to identify I and O with $\mathbb{R}^n$ for some n . A **computational problem** is simply a subset $\pi \subseteq I \times O$. A **program** is simply a partial function $P : I \rightarrow O$. So $P(x)$ may be undefined for some x , denoted $P(x) \uparrow$; if $P(x)$ is defined, then we write $P(x) \downarrow$. We say P is a **partial algorithm** for π if for all $x \in I$, if $P(x) \downarrow$ then $(x, P(x)) \in \pi$. An **algorithm** A for π is a partial algorithm that happens to be a total function. A **filter** for π is a total program $F : I \times O \rightarrow \{0, 1\}$ such that $F(x, y) = 1$ implies $(x, y) \in \pi$. A **checker** C for π is a filter for π such that if $C(x, y) = 0$ then $(x, y) \notin \pi$. Thus, a checker is a filter, but not necessarily vice-versa. Finally, a **filtered program** for π is a pair (P, F) such that P is a total program and F is a filter for π . We view (P, F) as a new program P_F , such that on input x , if $F(x, P(x)) = 1$ then $P_F(x) = P(x)$; otherwise $P_F(x) \uparrow$. Thus P_F is a partial algorithm for π . We want to “anchor” a filtered program (P, F) for π with an algorithm A for π . The triple (P, F, A) would constitute a program $P_{F,A}$ for π : on any input x , we define $P_{F,A}(x) = P_F(x)$ if $P_F(x) \downarrow$, otherwise $P_{F,A}(x) = A(x)$.

Logically, the algorithm (P, F, A) is no different from A alone. It is the complexity considerations that motivates (P, F, A) . For, if $C_A(x)$ denotes the complexity of the algorithm A on input x , we have

$$C_{(P,F,A)}(x) = C_P(x) + C_F(x, P(x)) + \delta$$

where

$$\delta = \begin{cases} 0 & \text{if } F(x, P(x)) = 1, \\ C_A(x) & \text{else.} \end{cases}$$

The algorithm (P, F, A) may be more efficient than A especially if $C_A(x)$ is expensive, and the filter is so efficacious that most of the time, $\delta = 0$.

We consider the filtered program (P, F) for π because P is presumably some useful program for π . We can consider filter cascades: let F be a sequence

$$F = (F_1, \dots, F_n) \tag{8}$$

where each F_i is a filter for π . We call F a (n -level) **filter cascade** for π . We view F as a filter for π by defining $F(x, y) = \max_{i=1}^n F_i(x, y)$. Thus $F(x, y) = 1$ iff some $F_i(x, y) = 1$. In practice, we evaluate $F(x, y)$ by searching for the first i such that $F_i(x, y) = 1$; and otherwise output 0. Funke, et al [30,15] are among the first to exploit multi-level filter cascades. A filter cascade (8) in practice would have the property that each F_i is more “effective” but less efficient than F_{i-1} .

In EGC, the filters are usually **numerical filters**. These filters certify some property of a computed numerical value, typically its sign. This often amounts to computing some error bound, and comparing the computed value with this bound. When such filters aim to certifying machine floating-point arithmetic, we call them **floating-point filters**. Ultimately, filters are used for efficiency purposes in EGC, not for correctness purposes.

There are two main classifications of numerical filters: static or dynamic. Static filters are those that can be computed at compile time for the most part, and they incur a low overhead at runtime. However, static error bounds may be overly pessimistic and thus less effective. Dynamic filters exhibit opposite characteristics: they have higher runtime cost but are much more effective (i.e., fewer false rejections). We can have semi-static filters which combine both features.

Filters can be used at different levels of granularity: from individual machine operations (e.g., arithmetic operations for dynamic filters), to subroutines (e.g., geometric primitives) to algorithms (e.g. convex hull or meshing algorithm).

Computing upper bounds in machine arithmetic. In the implementation of numerical filters, we need to compute sharp upper bounds on numerical expressions. To be specific, suppose you have IEEE double values x and y and you want to compute an upper bound on $|z|$ where $z = xy$. How can you do this? We can compute

$$\tilde{z} \leftarrow |x| \odot |y|. \quad (9)$$

Here, $|\cdot|$ is done exactly by the IEEE arithmetic, but the multiplication $\odot$ is not exact. One aspect of IEEE arithmetic is that we can change the rounding modes [89]. So if we change the rounding mode to round towards $+\infty$, we will have $\tilde{z} \geq |z|$. Otherwise, we only know that $\tilde{z} = |z|(1 + \delta)$ where $|\delta| \leq \mathbf{u}$. Here $\mathbf{u} = 2^{-53}$ is the “unit of rounding” for the arithmetic. We will describe the way to use the rounding modes later, in the interval arithmetic section. So here, instead of relying on rounding modes, we further compute $\tilde{w}$ as follows:

$$\tilde{w} \leftarrow \tilde{z} \odot (1 + 4\mathbf{u}). \quad (10)$$

It is assumed that overflow and underflow do not occur during the computation of $\tilde{w}$.

Note that $1 + 4\mathbf{u} = 1 + 2^{-51}$ is exactly representable. Therefore, we know that $\tilde{w} = \tilde{z}(1 + 4\mathbf{u})(1 + \delta')$ for some δ satisfying $|\delta'| \leq \mathbf{u}$. Hence,

$$\begin{aligned} \tilde{w} &= z(1 + \delta)(1 + \delta')(1 + 4\mathbf{u}) \\ &\geq z(1 - 2\mathbf{u} + \mathbf{u}^2)(1 + 4\mathbf{u}) \\ &= z(1 + 2\mathbf{u} - 7\mathbf{u}^2 + 4\mathbf{u}^3) \\ &> z \end{aligned}$$

Note that if any of the operations $\oplus$, $\ominus$ or $\oslash$ is used in place of $\odot$ in (9), the same argument still shows that $\tilde{w}$ is an upper bound on the actual value.

We summarize this result:

LEMMA 1 *Let E be any rational numerical expression and let $\tilde{E}$ be the approximation to E evaluated using IEEE double precision arithmetic. Assume the input numbers in E are IEEE doubles and E has $k \geq 1$ operations.*

- (i) *We can compute an IEEE double value $\mathbf{MaxAbs}(E)$ satisfying the inequality $|E| \leq \mathbf{MaxAbs}(E)$, in $3k$ machine operations.*
- (ii) *If all the input values are positive, $2k$ machine operations suffice.*
- (iii) *The value $\tilde{E}$ is available as a side effect of computing $\mathbf{MaxAbs}(E)$, at the cost of storing the result.*

PROOF. In proof, we simply replace each rational operation in E by at most 3 machine operations: we count 2 flops to compute $\tilde{z}$ in equations (9), and 1 flop to compute $\tilde{w}$ in (10). In case the input numbers are non-negative, $\tilde{z}$ needs only 1 machine operation.

5.1 Static Filters

Fortune and Van Wyk [29] were the first to implement and quantify the efficacy of filters for exact geometric computation. Their filter was implemented via the LN preprocessor system. We now look at the simple filter they implemented (which we dub the “FvW Filter”), and some of their experimental results.

The FvW filter. Static error bounds are easily maintained for a polynomial expression E with integer values at the leaves. Let $\tilde{E}$ denote the IEEE double value obtained by direct evaluation of E using IEEE double operations. Fortune and Van Wyk compute a bound $\text{MaxErr}(E)$ on the absolute error,

$$|E - \tilde{E}| \leq \text{MaxErr}(E). \quad (11)$$

It is easy to use this bound as a filter to certify the sign of $\tilde{E}$: if $|\tilde{E}| > \text{MaxErr}(E)$ then $\text{sign}(\tilde{E}) = \text{sign}(E)$. Otherwise, we must resort to some fall back action. Unless we specify otherwise, the default action is to immediately use an infallible method, namely computing exactly using a Big Number package.

Let us now see how to compute $\text{MaxErr}(E)$. It turns out that we also need the magnitude of E . The base-2 logarithm of the magnitude is bounded by $\text{MaxLen}(E)$. Thus, we say that the FvW filter has two **filter parameters**,

$$\text{MaxErr}(E), \quad \text{MaxLen}(E). \quad (12)$$

We assume that each input variable x is assigned an upper bound $\text{MaxLen}(x)$ on its bit length. Inductively, if F and G are polynomial expressions, then $\text{MaxLen}(E)$ and $\text{MaxErr}(E)$ are defined using the rules in Table 9.

Observe that the formulas in Table 9 assume exact arithmetic. In implementations, we have to be careful to compute upper bounds on these formulas. We assume that the filter has failed in case of an overflow; it is easy to see that no underflow occurs when evaluating these formulas. Checking for exceptions has an extra overhead. Since $\text{MaxLen}(E)$ is an integer, we can evaluate the corresponding formulas using IEEE arithmetic exactly. But the formulas for $\text{MaxErr}(E)$ will incur error, and we need to use some form of lemma 1.

Expr E	$\text{MaxLen}(E)$	$\text{MaxErr}(E)$
Var x	$\text{MaxLen}(x)$ given	$\max\{0, 2^{\text{MaxLen}(E)-53}\}$
$F \pm G$	$1 + \max\{\text{MaxLen}(F), \text{MaxLen}(G)\}$	$\text{MaxErr}(F) + \text{MaxErr}(G)$ $+ 2^{\text{MaxLen}(F \pm G)-53}$
FG	$\text{MaxLen}(F) + \text{MaxLen}(G)$	$\text{MaxErr}(F)2^{\text{MaxLen}(G)}$ $+ \text{MaxErr}(G)2^{\text{MaxLen}(F)}$ $+ 2^{\text{MaxLen}(FG)-53}$

Table 9

Parameters for the FvW filter

Framework for measuring filter efficacy. We want to quantify the efficacy of the FvW Filter. Consider the primitive of determining the sign of a 4×4 integer determinant. First look at the unfiltered performance of this primitive. We use the IEEE machine double arithmetic evaluation of this determinant (with possibly incorrect sign) as the **base line** for speed; this is standard procedure. This base performance is then compared to the performance of some standard (off-the-shelf) Big Integer packages. This serves as the **top line** for speed. The numbers cited in the paper are for the Big Integer package in LEDA (circa 1995), but the general conclusion for other packages are apparently not much different. For random 31-bit integers, the top line time yields 60 time increase over the base line. We will say

$$\sigma = 60 \tag{13}$$

in this case; the symbol σ (or $\sigma(31)$) reminds us that this is the “slowdown” factor. Clearly, σ is a function of the bit length L as well. For instance, with random 53-bit signed integers, the factor σ becomes 100. Next, still with $L = 31$, but using static filters implemented in LN, the factor σ ranges from 13.7 to 21.8, for various platforms and CPU speeds [29, Figure 14]. For simplicity, we simply say $\sigma = 20$, for some mythical combination of these platforms and CPUs. Thus the static filters improve the performance of exact arithmetic by the factor

$$\phi = 60/20 = 3. \tag{14}$$

In general, using unfiltered exact integer arithmetic as base line, the symbol ϕ denotes the “filtered improvement”. We use it as a measure of the efficacy of filtering.

In summary, the above experimental framework is clearly quite general, and aims to reduce the efficacy of a particular filter by estimating a number ϕ . In general, the framework requires the following choices: (1) a “test algorithm”

(we picked one for 4×4 determinants), (2) the “base line” (the standard is IEEE double arithmetic), (3) the “top line” (we picked LEDA’s Big Integer), (4) the input data (we consider random 31-bit integers). Its simplicity means that we can extract this ϕ factor from almost any⁶ published experimental papers. Another measure of efficacy is the fraction ρ of approximate values $\tilde{E}$ which fail to pass the filter. In [22], a general technique for assessing the efficacy of an arithmetic filter is proposed based on an analysis which consists of evaluating both the threshold value and the probability of failure of the filter.

For a true complexity model, we need to introduce size parameters. In EGC, two size parameters are of interest: the combinatorial size n and the bit size L . Hence all these parameters ought to be written as $\sigma(n, L)$, $\phi(n, L)$, etc.

Realistic versus synthetic problems. Static filters have an efficacy factor $\phi = 3$ (see (14)) in evaluating the sign of randomly generated 4-dimensional matrices ($L = 31$). Such problems are called “synthetic benchmarks” in [15]. So it would be interesting to see the performance of filters using “realistic benchmarks”, meaning actual algorithms for natural problems that we want to solve. But even here, there are degrees of realism. Let us⁷ equate realistic benchmarks with algorithms for problems such as convex hulls, triangulations, etc.

The point of realistic benchmarks is that they will generally involve a significant amount of non-numeric computation. Hence the ϕ -factor in such settings may look better than our synthetic ones. To quantify this, suppose that a fraction

$$\beta \quad (0 \leq \beta \leq 1) \tag{15}$$

of the running time of the algorithm is attributable to numerical computation. After replacing the machine arithmetic with exact integer arithmetic, the overall time becomes $(1 - \beta) + \beta\sigma = 1 + (\sigma - 1)\beta$. With filtered arithmetic, the time becomes $1 + (\sigma - 1)\beta\phi^{-1}$. Thus efficacy factor ϕ' for the algorithm is really

$$\phi' := \frac{(1 + (\sigma - 1)\beta)}{1 + (\sigma - 1)\beta/\phi}.$$

It is easy to verify that $\phi' < \phi$ (since $\phi > 1$).

⁶ Alternatively, we could require these numbers as the minimum standard for any experimental paper on filters.

⁷ While holding on tightly to our theoretician’s hat!

Note that our derivation assumes the original time is unit! This normalization is valid in our derivation because all the factors σ, ϕ that we use are ratios and are not affected by the normalization.

The factor β is empirical, of course. But even so, how can we estimate this? For instance, for 2- and 3-dimensional Delaunay triangulations, Fortune and Van Wyk [29] noted that $\beta \in [0.2, 0.5]$. Burnikel, Funke and Schirra [15] suggest a very simple idea for obtaining β : simply execute the test program in which each arithmetic operation is repeated $c > 1$ times. This gives us a new timing for the test program,

$$T(c) = (1 - \beta) + c\beta.$$

Now, by plotting the running time $T(c)$ against c , we obtain β as the slope.

Some detailed experiments on 3D Delaunay triangulations have been made by Devillers and Pion [21], comparing different filtering strategies, and conclude that cascading predicates is the best scheme in practice for this problem. Other experiments on interval arithmetic have been done by Seshia, Blleloch and Harper [79].

5.2 *Dynamic Filters*

To improve the quality of the static filters, we can use runtime information about the actual values of the variables, and dynamically compute the error bounds. We can again use $\text{MaxErr}(E)$ and $\text{MaxLen}(E)$ as found in Table 9 for static error. The only difference lies in the base case: for each variable x , the $\text{MaxErr}(x)$ and $\text{MaxLen}(x)$ can be directly computed from the value of x .

Dynamic version of the FvW filter. It is easy enough to convert the FvW filter into a dynamic one: looking at Table 9, we see that the only modification is that in the base case, we can directly compute $\text{MaxLen}(x)$ and $\text{MaxErr}(x)$ for a variable x . Let us estimate the cost of this dynamic filter. We already note that $\text{MaxLen}(E)$ can be computed directly using the formula in Table 9 since they involve integer arithmetic. This takes 1 or 2 operations. But for $\text{MaxErr}(E)$, we need to appeal to lemma 1. It is easy to see that since the values are all non-negative, we at most double the operation counts in the formulas of Table 9. The worst case is the formula for $E = FG$:

$$\text{MaxErr}(E) = \text{MaxErr}(F)2^{\text{MaxLen}(G)} + \text{MaxErr}(G)2^{\text{MaxLen}(F)} + 2^{\text{MaxLen}(FG)-53}.$$

The value $2^{\text{MaxLen}(FG)-53}$ can be computed exactly in 2 flops. There remain 4 other exact operations, which require $2 \times 4 = 8$ flops. Hence the bound here is 10 flops. Added to the single operation to compute $\text{MaxLen}(F) + \text{MaxLen}(G)$, we obtain 11 flops. A similar analysis for $E = F + G$ yields 8 flops.

After computing these filter parameters, we need to check if the filter predicate is satisfied:

$$|\tilde{E}| > \text{MaxErr}(E).$$

Assuming $\tilde{E}$ is available (this may incur storage costs not counted above), this check requires up to 2 operations: to compute the absolute value of $\tilde{E}$ and to perform the comparison. Alternatively, if $\tilde{E}$ is not available, we can replace $\tilde{E}$ by $2^{\text{MaxLen}(E)}$. In general, we also need to check for floating-point exceptions at the end of computing the filter parameters (the filter is assumed to have failed when an exception occurred). We may be able to avoid the exception handling, e.g., static analysis may tell us that no exceptions can occur.

Fortune and Van Wyk gave somewhat similar numbers, which we quote: let f_e count the extra runtime operations, including comparisons; let f_r count the runtime operations for storing intermediate results. In the LN implementation, $12 \leq f_e \leq 14$ and $24 \leq f_r \leq 33$. The $36 \leq f_e + f_r \leq 47$ overhead is needed even when the filter successfully certifies the approximate result $\tilde{E}$; otherwise, we may have to add the cost of exact computation, etc. Hence $f_e + f_r$ is a lower bound on the σ -factor when using filtered arithmetic in LN. Roughly the same bound of $\sigma = 48$ was measured for LEDA's system.

Other ideas can be brought into play: we need not immediately invoke the dynamic filter. We still maintain a static filter, and do the more expensive runtime filter only when the static filter fails. And when the dynamic filter fails, we may still resort to other less expensive computation instead of jumping immediately to some failsafe but expensive big Integer/Rational computation. The layering of these stages of computations is called **cascaded filtering** by Burnikel, Funke and Schirra (BFS) [13]. This technique seems to pay off especially in degenerate situations. We next describe the BFS filter.

The BFS filter. This is a dynamic filter, but it can also be described as “semi-static” (or “semi-dynamic”) because one of its two computed parameters is statically determined. Let E be a radical expression, *i.e.*, involving $+$, $-$, $\times$, $\div$, $\sqrt{}$. Again, let $\tilde{E}$ be the machine IEEE double value computed from E in the straightforward manner (this time, with division and square-roots). In contrast to the FvW Filter, the filter parameters are now

$$\text{MaxAbs}(E), \quad \text{Ind}(E).$$

The first is easy to understand: $\text{MaxAbs}(E)$ is simply an upper bound on $|E|$. The second, called the **index** of E , is a natural number whose interpretation can roughly be that its base 2 logarithm is roughly the number of bits of precision which are lost (*i.e.* which the filter cannot guarantee) at best in the evaluation of the expression. Together, they satisfy the following invariant:

$$|E - \tilde{E}| \leq \text{MaxAbs}(E) \cdot \text{Ind}(E) \cdot 2^{-53} \tag{16}$$

Of course, the value 2^{-53} may be replaced by the unit roundoff error $\mathbf{u}$ in general for other floating-point systems.

Table 10 gives the recursive rules for maintaining $\mathbf{MaxAbs}(E)$ and $\mathbf{Ind}(E)$. The base case (E is a variable) is covered by the first two rows: notice that they distinguish between exact and rounded input variables. A variable x is **exact** if its value is representable without error by an IEEE double. In any case, x is assumed not to lie in the overflow range, so that the following holds

$$|\mathbf{round}(x) - x| \leq |x|2^{-53}.$$

Also note that the bounds are computed using IEEE machine arithmetic, denoted

$$\oplus, \ominus, \odot, \oslash, \sqrt{\cdot}.$$

The question arises: what happens when the operations lead to over- or underflow in computing the bound parameters? It can be shown that underflows for $\oplus$, $\ominus$ and $\sqrt{\cdot}$ can be ignored, and in the case of $\odot$ and $\oslash$, we just have to add a small constant $\mathit{MinDbl} = 10^{-1022}$ to $\mathbf{MaxAbs}(E)$.

Expression E	$\mathbf{MaxAbs}(E)$	$\mathbf{Ind}(E)$
Exact var. x	x	0
Approx. var. x	$\mathbf{round}(x)$	1
$E = F \pm G$	$\mathbf{MaxAbs}(F) \oplus \mathbf{MaxAbs}(G)$	$1 + \max\{\mathbf{Ind}(F), \mathbf{Ind}(G)\}$
$E = FG$	$\mathbf{MaxAbs}(F) \odot \mathbf{MaxAbs}(G)$	$1 + \mathbf{Ind}(F) + \mathbf{Ind}(G)$
$E = F/G$	$\frac{ \tilde{E} \oplus (\mathbf{MaxAbs}(F) \odot \mathbf{MaxAbs}(G))}{(G \odot \mathbf{MaxAbs}(G)) \ominus (\mathbf{Ind}(G) + 1)2^{-53}}$	$1 + \max\{\mathbf{Ind}(F), \mathbf{Ind}(G) + 1\}$
$E = \sqrt{F}$	$\begin{cases} (\mathbf{MaxAbs}(F) \odot \tilde{F}) \odot \tilde{E} & \text{if } \tilde{F} > 0 \\ \sqrt{\mathbf{MaxAbs}(F)} \odot 2^{26} & \text{if } \tilde{F} = 0 \end{cases}$	$1 + \mathbf{Ind}(F)$

Table 10

Parameters of the BFS filter

Under the assumption (16), we can use the following criteria for certifying the sign of $\tilde{E}$:

$$|\tilde{E}| > \mathbf{MaxAbs}(E) \cdot \mathbf{Ind}(E) \cdot 2^{-53} \quad (17)$$

Of course, this criteria should be implemented using machine arithmetic (see (10) and notes there). One can even certify the exactness of $\tilde{E}$ under certain conditions. If E is a polynomial expression (*i.e.*, involving $+$, $-$, $\times$ only), then $E = \tilde{E}$ provided

$$1 > \mathbf{MaxAbs}(E) \cdot \mathbf{Ind}(E) \cdot 2^{-52}. \quad (18)$$

Finally, we look at some experimental results. Table 11 shows the σ -factor (recall that this is a slowdown factor compared to IEEE machine arithmetic) for the unfiltered and filtered cases. In both cases, the underlying Big Integer package is from LEDA. The last column adds compilation to the filtered case. It is based on an expression compiler, EXPCOMP, somewhat in the spirit of LN (see 5.3). At $L = 32$, the ϕ -factor (recall this is the speedup due to filtering) is $65.7/2.9 = 22.7$. When compilation is used, it improves to $\phi = 65.7/1.9 = 34.6$. [Note: the reader might be tempted to deduce from these numbers that the BFS filter is more efficacious than the FvW Filter. But the use of different Big Integer packages, platforms and compilers, etc, does not justify this conclusion.]

BitLength L	Unfiltered σ	BFS Filter σ	BFS Compiled σ
8	42.8	2.9	1.9
16	46.2	2.9	1.9
32	65.7	2.9	1.9
40	123.3	2.9	1.8
48	125.1	2.9	1.8

Table 11
Random 3×3 determinants

While the above results look good, it is possible to create situations where filters are ineffective. Instead of using matrices with randomly generated integer entries, we can use degenerate determinants as input. The results recorded in Table 12 indicate that filters have very little effect. Indeed, we might have expected it to slow down the computation, since the filtering effort are strictly extra overhead. In contrast, for random data, the filter is almost always effective in avoiding exact computation.

BitLength L	Unfiltered σ	BFS Filter σ	BFS Compiled σ
8	37.9	2.4	1.4
16	45.3	2.4	1.4
32	56.3	56.5	58.4
40	117.4	119.4	117.5
48	135.2	136.5	135.1

Table 12
Degenerate 3×3 determinants

The original paper [13] describes more experimental results, including the performance of the BFS filter in the context of algorithms for computing Voronoi diagrams and triangulation of simple polygons.

Dynamic filter using interval arithmetic As mentionned previously, a simpler and more traditionnal way to control the error made by floating-point computations is to use interval arithmetic [62,1]. Some work has been done by Pion et al [69,70,9] in this direction.

The foundation of interval arithmetic is the representation of the error bound on an expression E at runtime by an interval $[E_m; E_p]$ whose bounds are floating-point values, and which contains E . So, for a given predicate evaluation, intervals start in the form $[x; x]$ when the input is exactly representable as a double value, and tend to grow after some operations, accounting for the error.

Each arithmetic operation $+, -, \times, \div, \sqrt{}$ performed on intervals preserve the inclusion property, that is, for each real value(s) contained in the operand interval(s), the real result of the corresponding arithmetic operation will be certified to belong to the resulting interval. Technically, this is usually achieved by relying on the IEEE rounding modes.

It is clear that changing the rounding mode has a certain cost (mostly due to flushing the pipeline of the FPU), but the remark has been made that it can usually be done only twice per predicate : at the beginning by setting the rounding mode towards $+\infty$, and at the end to reset it back to the default mode. This can be achieved by observing that computing $a+b$ rounded towards $-\infty$ can be emulated by computing $-((-a) - b)$ rounded towards $+\infty$. A similar remark can be done for $-, \times, \div$. Therefore it is possible to eliminate most rounding mode changes, which makes the approach much more efficient.

Most experimental studies (e.g. [21,79]) show that using interval arithmetic implemented this way usually induces a slowdown factor of 3 to 4 on algorithms, compared to floating-point. It is also to be noted that interval arithmetic is the most efficacious dynamic filter, failing rarely. This technic is available in the CGAL library, covering all predicates of the geometry kernel.

5.3 *Tools for automatic generation of code for the filters*

Given the algebraic formula for a predicate, it is boring and error-prone to derive the filtered version of this predicate manually, especially when there are lots of them. Therefore some tools have been developed to help generating these codes.

We have already mentionned the first one, LN, which targets the FvW filter [29]. This tool helps generating code, but is limited when trying more complex predicates, which include divisions, square roots, branches or loops.

Another attempt has been made by Funke et al [14] with a tool called EXPCOMP (standing for expression compiler), which parses slightly modified C++ code of the original predicate, and produces static and semi-static BFS filters for them, as we already mentioned.

The **CGAL** library implements filtering using interval arithmetic for all the predicates in its geometry kernel. The filtered versions of these predicates are generated by a Perl script [69,70], which can be applied by the user to his own predicates. The advantage of dynamic filters such as this one is that the code generator does not have to find the structure of the predicate, and so it makes writing a tool an easier task. The template mechanism of C++ is also used in this context to prevent too much code duplication.

Most recently, Nanevski, Blleloch and Harper [64] have proposed a tool that produces filters using Shewchuk's method [80], for the SML language, from an SML code of the predicate. They also have the possibility to produce C code for this method.

Seeing these past and ongoing works, it seems important to have a general tool to handle this kind of numerical problems, which are connected to compiler technology and static code analysis.

6 Conclusions

Among existing approaches to non-robustness, Exact Geometric Computation shows its distinctive advantages in its generality and ease-of-use. The presence of the major software libraries such as **LEDA** and **CGAL** based on EGC, testifies to its utility. We discussed some key efficiency issues. The sign determination problem and the role of constructive root bounds is basic. When carefully combined with techniques such as precision-driven computation and filtering, the efficiency of EGC can be greatly improved.

Through EGC, numerical non-robustness for a large class of geometric computation (low dimensions and low degree problems) has been brought out of the realm of "unsolved" to the realm of "solved but practically challenging". For these problems, in addition to the above topics, we believe that compiler-like techniques will help us solve these problems routinely in the future. Filter technology itself can be developed into a profoundly interesting subject on its own right, including making connections to program checking.

For high degree problems, efficiency will be a serious challenge. But this seems inherent rather than a failing of the EGC approach. Here we must rely on progress in root bounds methods, problem specific techniques, and perhaps

completely new ideas. For non-algebraic problems, the fundamental question whether they even admit EGC solutions is completely open and relates to some deep questions in mathematics.

Reliable geometric computation can find direct applications in many areas, including geometric modeling, CAD and robotics. EGC has so far been applied at the level of stand-alone algorithms. Applying EGC in a real and complex application like CAD system would be a challenge. Then the problem of geometric rounding should be addressed in greater earnest.

Another real world issue which we have ignored is how to treat inexact inputs. For simple applications, we can treat them as “nominally exact”. But if this can lead to inconsistent inputs, we basically need to treat more complex classes of geometric objects (e.g., we now think of points as balls) and new semantics. But research on this topic has not yet begun. We have begun to explore the notion of robustness as a computational resource [42]. That is, we no longer view robustness as a 0-1 proposition. Instead, we want to compute at various points of some implicit speed-robustness tradeoff curve. In some sense, this is simply acknowledging the lay man’s view of non-robustness. The difficulty lies in building a model for this.

In conclusion, we have reasons to be optimistic that EGC concepts will become more and more a part of the computing landscape. The inexorable logic of Moore’s law implies that more and more applications will no longer be speed-critical, and thus able to use the robust, if slower, EGC solutions.

References

- [1] G. Alefeld and J. Herzberger. *Introduction to Interval Computation*. Academic Press, New York, 1983.
- [2] R. L. Ashenhurst and N. Metropolis. Error estimates in computer calculation. *Amer. Math. Monthly*, 72(2):47–58, 1965.
- [3] M. Benouamer, P. Jaillon, D. Michelucci, and J.-M. Moreau. A lazy arithmetic library. In *Proceedings of the IEEE 11th Symposium on Computer Arithmetic*, pages 242–269, Windsor, Ontario, June 30-July 2, 1993.
- [4] M. Benouamer, D. Michelucci, and B. Péroche. Boundary evaluation using a lazy rational arithmetic. In *Proceedings of the 2nd ACM/IEEE Symposium on Solid Modeling and Applications*, pages 115–126, Montréal, Canada, 1993. ACM Press.
- [5] J. Blömer. A probabilistic zero-test for expressions involving roots of rational numbers. *Proc. of the Sixth Annual European Symposium on Algorithms*, pages 151–162, 1998. LNCS 1461.

- [6] J. Blömer. *Simplifying Expressions Involving Radicals*. PhD thesis, Free University Berlin, Department of Mathematics, October, 1992.
- [7] G. Bohlender, C. Ullrich, J. W. von Gudenberg, and L. B. Rall. *Pascal-SC*, volume 17 of *Perspectives in Computing*. Academic Press, Boston-San Diego-New York, 1990.
- [8] J.-D. Boissonnat and M. Yvinec. *Algorithmic Geometry*. Cambridge University Press, 1997. Translated by Hervé Brönnimann.
- [9] H. Brönnimann, C. Burnikel, and S. Pion. Interval arithmetic yields efficient dynamic filters for computational geometry. *Discrete Applied Mathematics*, 109:25–47, 2001.
- [10] C. Burnikel. *Exact Computation of Voronoi Diagrams and Line Segment Intersections*. Ph.D thesis, Universität des Saarlandes, March 1996.
- [11] C. Burnikel, R. Fleischer, K. Mehlhorn, and S. Schirra. Exact geometric computation made easy. In *Proc. 15th ACM Symp. Comp. Geom.*, pages 341–450, 1999.
- [12] C. Burnikel, R. Fleischer, K. Mehlhorn, and S. Schirra. A strong and easily computable separation bound for arithmetic expressions involving radicals. *Algorithmica*, 27:87–99, 2000.
- [13] C. Burnikel, S. Funke, and M. Seel. Exact geometric predicates using cascaded computation. *Proceedings of the 14th Annual Symposium on Computational Geometry*, pages 175–183, 1998.
- [14] C. Burnikel, S. Funke, and M. Seel. Exact geometric computation using cascading. *Internat. J. Comput. Geom. Appl.*, 11:245–266, 2001.
- [15] S. Burnikel, Funke. Exact geometric computation using cascading. *to appear in special issue of IJCGA*, 2000.
- [16] J. F. Canny. *The complexity of robot motion planning*. ACM Doctoral Dissertation Award Series. The MIT Press, 1988. PhD thesis, M.I.T.
- [17] J. F. Canny. Some algebraic and geometric configurations in PSPACE. In *Proc. 20th Annu. ACM Sympos. Theory Comput.*, pages 460–467, 1988.
- [18] B. Chazelle et al. Application challenges to computational geometry. In *Advances in Discrete and Computational Geometry*, volume 223 of *Contemporary Mathematics*, pages 407–463. AMS, 1999. The Computational Geometry Impact Task Force Report (1996).
- [19] C. Clenshaw, F. Olver, and P. Turner. Level-index arithmetic: an introductory survey. In P. Turner, editor, *Numerical Analysis and Parallel Processing*, pages 95–168. Springer-Verlag, 1987. Lecture Notes in Mathematics, No.1397.
- [20] M. de Berg, M. van Kreveld, M. Overmars, and O. Schwarzkopf. *Computational Geometry: Algorithms and Applications*. Springer-Verlag, Berlin, 1997.

- [21] O. Devillers and S. Pion. Efficient exact geometric predicates for Delaunay triangulations. In *Proc. 5th Workshop Algorithm Eng. Exper.*, Jan. 2003. To appear.
- [22] O. Devillers and F. P. Preparata. A probabilistic analysis of the power of arithmetic filters. *Discrete Comput. Geom.*, 20:523–547, 1998.
- [23] T. Dubé and C. K. Yap. A basis for implementing exact geometric algorithms (extended abstract), September, 1993. Paper from <ftp://cs.nyu.edu/pub/local/yap/exact/basis.ps.gz>.
- [24] H. Edelsbrunner. *Algorithms in Combinatorial Geometry*. Springer-Verlag, 1987.
- [25] A. R. Forrest. Computational geometry and software engineering: Towards a geometric computing environment. In D. F. Rogers and R. A. Earnshaw, editors, *Techniques for Computer Graphics*, pages 23–37. Springer-Verlag, 1987.
- [26] S. Fortune. Introduction (editorial for special issue on implementation of geometric algorithms), 2000.
- [27] S. J. Fortune. Stable maintenance of point-set triangulations in two dimensions. *IEEE Foundations of Computer Science*, 30:494–499, 1989.
- [28] S. J. Fortune and C. J. van Wyk. Efficient exact arithmetic for computational geometry. In *Proc. 9th ACM Symp. on Computational Geom.*, pages 163–172, 1993.
- [29] S. J. Fortune and C. J. van Wyk. Static analysis yields efficient exact integer arithmetic for computational geometry. *ACM Transactions on Graphics*, 15(3):223–248, 1996.
- [30] S. Funke. Exact arithmetic using cascaded computation. Master’s thesis, Max Planck Institute for Computer Science, Saarbrücken, Germany, 1997.
- [31] D. Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991.
- [32] A. J. Goldstein and R. L. Graham. A Hadamard-type bound on the coefficients of a determinant of polynomials. *SIAM Review*, 16:394–395, 1974.
- [33] J. E. Goodman and J. O’Rourke, editors. *Handbook of Discrete and Computational Geometry*. CRC Press LLC, 1997.
- [34] M. Goodrich, L. Guibas, J. Hershberger, and P. Tanenbaum. Snap rounding line segments efficiently in two and three dimensions. In *Proc. 13th ACM Symp. on Computational Geom.*, pages 284–293, 1997.
- [35] D. H. Greene and F. F. Yao. Finite-resolution computational geometry. *IEEE Foundations of Computer Science*, 27:143–152, 1986.
- [36] L. Guibas and D. Marimont. Rounding arrangements dynamically. In *Proc. 11th ACM Symp. Computational Geom.*, pages 190–199, 1995.

- [37] L. Guibas, D. Salesin, and J. Stolfi. Epsilon geometry: building robust algorithms from imprecise computations. *ACM Symp. on Computational Geometry*, 5:208–217, 1989.
- [38] J. Hobby. Practical segment intersection with finite precision output. Technical report, Bell Labs, 1993. Tech. Report.
- [39] J. D. Hobby. Practical segment intersection with finite precision output. *Comput. Geom. Theory Appl.*, 13(4):199–214, Oct. 1999.
- [40] C. Hoffmann, J. Hopcroft, and M. Karasick. Towards implementing robust geometric computations. *ACM Symp. on Computational Geometry*, 4:106–117, 1988.
- [41] C. M. Hoffmann. The problems of accuracy and robustness in geometric computation. *IEEE Computer*, 22(3), March 1989.
- [42] Exact Geometric Computation Homepage, 1996. FAQs, downloads, documentation, and related links available from the URL <http://cs.nyu.edu/exact/>.
- [43] CGAL Homepage, 1998. Computational Geometry Algorithms Library (CGAL) Project. A 7-institution European Community effort. See URL <http://www.cgal.org/>.
- [44] CORE Homepage, 1998. Core Library Project: URL <http://cs.nyu.edu/exact/core/>.
- [45] LEDA Homepage, 1998. Library of Efficient Data Structures and Algorithms (LEDA) Project. From the Max Planck Institute of Computer Science. See URL <http://www.mpi-sb.mpg.de/LEDA/>.
- [46] G. Horng and M. D. Huang. Simplifying nested radicals and solving polynomials by radicals in minimum depth. *Proc. 31st Symp. on Foundations of Computer Science*, pages 847–854, 1990.
- [47] T. Hull and M. Cohen. Toward an ideal computer arithmetic. In *Proceedings of the 8th Symposium on Computer Arithmetic*, pages 5–48. IEEE, 1987.
- [48] V. Karamcheti, C. Li, I. Pechtchanski, and C. Yap. A Core library for robust numerical and geometric libraries. In *15th ACM Symp. Computational Geometry*, pages 351–359, 1999.
- [49] M. Karasick, D. Lieber, and L. R. Nackman. Efficient Delaunay triangulation using rational arithmetic. *ACM Trans. on Graphics*, 10:71–91, 1991.
- [50] S. Landau. Simplification of nested radicals. *SIAM Journal of Computing*, 21(1):85–110, 1992.
- [51] C. Li. *Exact Geometric Computation: Theory and Applications*. Ph.d. thesis, Department of Computer Science, New York University, Jan. 2001. Download from <http://cs.nyu.edu/exact/doc/>.

- [52] C. Li and C. Yap. A new constructive root bound for algebraic expressions. In *Proceedings of the Twelfth ACM-SIAM Symposium on Discrete Algorithms (SODA 2001)*, pages 496–505, Jan. 2001.
- [53] M. C. Lin and D. Manocha, editors. *Proceedings of the First ACM Workshop on Applied Computational Geometry*, 1996.
- [54] G. Liotta, F. Preparata, and R. Tamassia. Robust proximity queries: an illustration of degree-driven algorithm design. *ACM Symp. on Computational Geometry*, 13:156–165, 1997.
- [55] M. Marden. *The geometry of the zeros of a polynomial in a complex variable*. American Mathematical Society, 1949.
- [56] S. Matsui and M. Iri. An overflow/underflow-free floating-point representation of numbers. *J. Inform. Process*, 4(3):123–133, 1981.
- [57] K. Mehlhorn and S. Schirra. A generalized and improved constructive separation bound for real algebraic expressions. Technical Report MPI-I-2000-004, Max-Planck-Institut für Informatik, Nov. 2000.
- [58] N. Metropolis. Methods of significance arithmetic. In D. A. H. Jacobs, editor, *The State of the Art in Numerical Analysis*, pages 179–192. Academic Press, London, 1977.
- [59] M. Mignotte and D. Ștefănescu. *Polynomials: An Algorithmic Approach*. Springer, 1999.
- [60] V. Milenkovic and L. Nackman. Finding compact coordinate representations for polygons and polyhedra. *ACM Symp. on Computational Geometry*, 6:244–252, 1990.
- [61] V. J. Milenkovic. Verifiable implementations of geometric algorithms using finite precision arithmetic. *Artificial Intelligence*, 37:377–401, 1988. An earlier version appeared in *Proceedings, Oxford Workshop on Geometric Reasoning*, (eds. Brady, Hopcroft, Mundy).
- [62] R. E. Moore. *Interval Analysis*. Prentice Hall, Englewood Cliffs, NJ, 1966.
- [63] K. Mulmuley. *Computational Geometry: an Introduction through Randomized Algorithms*. Prentice-Hall, Inc, Englewood Cliffs, New Jersey, 1994.
- [64] A. Nanevski, G. Blelloch, and R. Harper. Automatic generation of staged geometric predicates. In *International Conference on Functional Programming*, Florence, Italy, 2001. Also Carnegie Mellon CS Tech Report CMU-CS-01-141.
- [65] J. O’Rourke. *Computational Geometry in C*. Cambridge University Press, second edition edition, 1998.
- [66] T. Ottmann, G. Thieme, and C. Ullrich. Numerical stability of geometric algorithms. In *Proc. 3rd ACM Sympos. Comput. Geom.*, pages 119–125, 1987.

- [67] K. Ouchi. Real/Expr: Implementation of an exact computation package. Master's thesis, New York University, Department of Computer Science, Courant Institute, January 1997. Download from <http://cs.nyu.edu/exact/doc/>.
- [68] N. M. Patrikalakis, W. Cho, C.-Y. Hu, T. Maekawa, E. C. Sherbrooke, and J. Zhou. Towards robust geometric modelers, 1994 progress report. In *Proc. 1995 NSF Design and Manufacturing Grantees Conference*, pages 139–140, 1995.
- [69] S. Pion. *De la géométrie algorithmique au calcul géométrique*. Thèse de doctorat en sciences, Université de Nice-Sophia Antipolis, France, 1999. TU-0619.
- [70] S. Pion. Interval arithmetic: An efficient implementation and an application to computational geometry. In *Workshop on Applications of Interval Analysis to systems and Control*, pages 99–110, 1999.
- [71] F. P. Preparata and M. I. Shamos. *Computational Geometry*. Springer-Verlag, 1985.
- [72] S. M. Rump. Polynomial minimum root separation. *Math. Comp.*, 33:327–336, 1979.
- [73] E. R. Scheinerman. When close enough is close enough. *Amer. Math. Monthly*, 107:489–499, 2000.
- [74] S. Schirra. Robustness and precision issues in geometric computation. In J.-R. Sack and J. Urrutia, editors, *Handbook of Computational Geometry*, chapter 14, pages 597–632. Elsevier Science Publishers B.V. North-Holland, Amsterdam, 2000.
- [75] P. Schorn. An axiomatic approach to robust geometric programs. *J. of Symbolic Computation*, 16:155–165, 1993.
- [76] J. T. Schwartz. Polynomial minimum root separation (Note to a paper of S. M. Rump). Technical Report 39, Courant Institute of Mathematical Sciences, Robotics Laboratory, New York University, Feb. 1985.
- [77] M. G. Segal and C. H. Sequin. Consistent calculations for solids modelling. In *Proc. 1st ACM Sympos. Comput. Geom.*, pages 29–38, 1985.
- [78] H. Sekigawa. Using interval computation with the Mahler measure for zero determination of algebraic numbers. *Josai Information Sciences Researches*, 9(1):83–99, 1998.
- [79] S. A. Seshia, G. E. Blelloch, and R. W. Harper. A performance comparison of interval arithmetic and error analysis in geometric predicates. Technical Report CMU-CS-00-172, School of Computer Science, Carnegie-Mellon University, 2000.
- [80] J. R. Shewchuk. Robust adaptive floating-point geometric predicates. In *Proc. 12th ACM Symp. on Computational Geom.*, pages 141–150. Association for Computing Machinery, May 1996.

- [81] K. Sugihara. An intersection algorithm based on Delaunay triangulation. *IEEE Computer Graphics Appl.*, 12(2):59–67, 1992.
- [82] K. Sugihara and M. Iri. A solid modeling system free from topological inconsistency. *J.Information Processing, Information Processing Society of Japan*, 12(4):380–393, 1989.
- [83] K. Sugihara and M. Iri. Two design principles of geometric algorithms in finite precision arithmetic. *Applied Mathematics Letters*, 2:203–206, 1989.
- [84] K. Sugihara and M. Iri. Construction of the Voronoi diagram for ‘one million’ generators in single-precision arithmetic. *Proc. IEEE*, 80(9):1471–1484, Sept. 1992.
- [85] K. Sugihara and M. Iri. An approach to the problem of numerical errors in geometric algorithms. *Proc., 37th Convention of the Information Processing Society of Japan, Kyoto*, pages 1665–1666, September 12–14, 1988.
- [86] K. Sugihara and M. Iri. Geometric algorithms in finite-precision arithmetic. Research Memorandum RMI 88-10, Dept. of Math. Engineering and Instrumentation Physics, Faculty of Engineering, University of Tokyo, September, 1988. 13th International Symposium on Mathematical Programming, Tokyo, Aug 29–Sep 2, 1988.
- [87] K. Sugihara, M. Iri, H. Inagaki, and T. Imai. Topology-oriented implementation—an approach to robust geometric algorithms. *Algorithmica*, 27:5–20, 2000.
- [88] R. Tamassia, P. Agarwal, N. Amato, D. Chen, D. Dobkin, R. Drysdale, S. Fortune, M. Doorich, J. Hersberger, J. O’Rourke, F. Preparata, J.-R. Sack, S. Suri, I. Tollis, J. Vitter, and S. Whitesides. Strategic directions in computational geometry working group report. *ACM Computing Surveys*, 28(4), Dec. 1996.
- [89] The Institute of Electrical and Electronic Engineers, Inc. IEEE Standard 754-1985 for binary floating-point arithmetic, 1985. ANSI/IEEE Std 754-1985. Reprinted in SIGPLAN 22(2) pp. 9-25.
- [90] D. Tulone, C. Yap, and C. Li. Randomized zero testing of radical expressions and elementary geometry theorem proving. In J. Richter-Gebert and D. Wang, editors, *Proc. 3rd Int’l. Workshop on Automated Deduction in Geometry (ADG’00)*, number 2061 in Lecture Notes in Artificial Intelligence, pages 58–82. Springer, 2001. Zurich, Switzerland.
- [91] C. Yap. A new number core for robust numerical and geometric libraries. In *3rd CGC Workshop on Geometric Computing*, 1998. Invited Talk. Brown University, Oct 11–12, 1998. For abstracts, see <http://www.cs.brown.edu/cgc/cgc98/home.html>.
- [92] C. Yap and C. Li. Core Library Tutorial: a library for robust geometric computation, 1999. Released with the Core Library software package, 1999–2001. Download: <http://cs.nyu.edu/exact/core/>.

- [93] C. K. Yap. Robust geometric computation. In J. E. Goodman and J. O'Rourke, editors, *Handbook of Discrete and Computational Geometry*, chapter 35, pages 653–668. CRC Press LLC, 1997.
- [94] C. K. Yap. Towards exact geometric computation. *Computational Geometry: Theory and Applications*, 7:3–23, 1997. Invited talk, Proceed. 5th Canadian Conference on Comp. Geometry, Waterloo, Aug 5–9, 1993.
- [95] C. K. Yap. *Fundamental Problems in Algorithmic Algebra*. Oxford Univ. Press, Dec. 1999.
- [96] C. K. Yap and T. Dubé. The exact computation paradigm. In D.-Z. Du and F. K. Hwang, editors, *Computing in Euclidean Geometry*, pages 452–486. World Scientific Press, Singapore, 1995. 2nd edition.
- [97] J. Yu. *Exact arithmetic solid modeling*. Ph.D. dissertation, Department of Computer Science, Purdue University, West Lafayette, IN 47907, June 1992. Technical Report No. CSD-TR-92-037.